



AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE

Wydział Inżynierii Mechanicznej i Robotyki

KATEDRA ROBOTYKI I MECHATRONIKI

Projekt dyplomowy

***Projekt i wykonanie systemu lokalizacji pojazdu
Design and Implementation of a Vehicle Localization
System***

Autor:	<i>Miłosz Stec</i>
Kierunek studiów:	Inżynieria Mechatroniczna
Opiekun pracy:	<i>dr inż. Grzegorz Góra</i>

Kraków, 2025/2026

Spis treści

1.	Wstęp	4
2.	Cel i zakres pracy.....	6
2.1.	Cele szczegółowe pracy:	6
2.2.	Zakres pracy obejmuje następujące etapy:	6
2.3.	Założenia projektowe	6
2.3.1.	Założenia funkcjonalne	7
2.3.2.	Założenia sprzętowe	7
2.3.3.	Założenia programowe	8
3.1.	Przegląd technologiczny i środowisko programistyczne.....	9
3.1.1.	Magistrala CAN (Controller Area Network).....	9
3.1.2.	Protokół MQTT (Message Queuing Telemetry Transport).....	10
3.1.3.	Systemy GNSS i standard NMEA 0183	11
3.1.4.	Magistrala 1-Wire (Interfejs Dallas).....	12
3.2.	Analiza wybranych rozwiązań rynkowych.....	13
3.2.1.	Lokalizator GPS BearPoint C18.....	14
3.2.2.	Lokalizator GPS OBD MK08.....	15
3.2.3.	Lokalizator GPS MK6B 4G (Magnetyczny)	16
3.2.4.	Kategoryzacja dostępnych rozwiązań.....	17
3.2.5.	Analiza porównawcza.....	18
3.2.6.	Wnioski z przeglądu i uzasadnienie wyboru technologii	19
4.	Projekt i implementacja urządzenia	20
4.1.	Architektura sprzętowa	20
4.1.1.	Jednostka centralna.....	20
4.1.2.	Moduł pozycjonowania GNSS.....	21
4.1.3.	Komunikacja z magistralą CAN.....	21
4.1.4.	Pomiar temperatury	22
4.1.5.	Pamięć masowa	22
4.1.6.	Układ zasilania	22
4.2.	Architektura oprogramowania	23
4.2.1.	Organizacja kodu źródłowego i struktura plików	23
4.2.2.	Podział na zadania (Tasks).....	24
4.2.3.	Użyte biblioteki	27
4.2.4.	Struktura danych i technika X-Macro	28
4.2.4.	Formaty danych.....	29
4.2.5.	Zaawansowana obsługa pamięci (SdModule).....	30

4.2.6. Mechanizmy synchronizacji i komunikacji międzyprocesowej.....	30
4.2.7. Zarządzanie danymi i tryb Offline	31
4.2.8. Synchronizacja czasu	31
4.3. Integracja z platformą ThingsBoard.....	31
4.3.1. Przetwarzanie danych i Silnik Reguł.....	31
4.3.2. Wizualizacja danych i monitoring.....	33
4.3.3. Dwukierunkowa komunikacja i zdalna konfiguracja.....	34
4.4 Wykonanie.....	35
4.5 Testy	39
4.5.1. Weryfikacja elektryczna i uruchomieniowa.....	40
4.5.2. Testy funkcjonalne oprogramowania	41
4.5.3. Testy polowe i ciągłość danych (Scenariusz Offline/Online)	43
4.5.4. Ocena jakości danych GNSS.....	44
4.5.5. Wizualizacja i telemetria	45
5. Podsumowanie i wnioski	46
5.1. Napotkane problemy i ograniczenia realizacyjne	46
5.2. Wnioski z realizacji	48
5.3. Kierunki dalszego rozwoju.....	48
6. Wykaz Publikacji.....	51
7. Załączniki.....	53

1. Wstęp

Dynamiczny rozwój technologii Internetu Rzeczy (IoT) oraz postępująca miniaturyzacja układów elektronicznych redefiniują współczesne podejście do systemów monitorujących. Pod pojęciem Internetu Rzeczy rozumie się koncepcję, w której przedmioty fizyczne (rzeczy) są wyposażone w czujniki, oprogramowanie i inne technologie w celu łączenia się i wymiany danych z innymi urządzeniami i systemami za pośrednictwem Internetu. Możliwość tworzenia zaawansowanych, a jednocześnie kompaktowych i energooszczędnych urządzeń znajduje szerokie zastosowanie w wielu gałęziach przemysłu, ze szczególnym uwzględnieniem logistyki i transportu.

Współczesne systemy lokalizacyjne ewoluowały z prostych rejestratorów pozycji do roli kompleksowych narzędzi telemetrycznych. Telemetria, będąca kluczowym elementem omawianego zagadnienia, to dziedzina techniki zajmująca się automatycznym pozyskiwaniem danych pomiarowych z trudno dostępnych miejsc oraz ich przesyłaniem drogą radiową lub przewodową do zewnętrznych systemów rejestrujących. Pozwalają one nie tylko na śledzenie trasy pojazdu, ale także na akwizycję i analizę danych diagnostycznych oraz parametrów środowiskowych, co znacząco zwiększa ich wartość użytkową w zarządzaniu flotą.

Niniejsza praca inżynierska dotyczy projektu i realizacji autorskiego systemu lokalizacji pojazdu, opartego na platformie sprzętowej ESP32. Opracowany układ, działający w architekturze IoT, ma za zadanie rejestrować dane o położeniu geograficznym i zapisywać je lokalnie na karcie pamięci SD, a po uzyskaniu dostępu do zaufanej sieci bezprzewodowej – synchronizować je z serwerem zewnętrznym. Projekt zakłada również opcjonalną integrację z magistralą pojazdu (CAN/OBD2) w celu rejestracji podstawowych parametrów eksploatacyjnych oraz możliwość rozbudowy o czujniki środowiskowe.

Główną motywacją do podjęcia tematu była chęć pogłębienia wiedzy w zakresie systemów wbudowanych czasu rzeczywistego oraz praktycznego zastosowania protokołów komunikacyjnych w systemach rozproszonych. Istotnym czynnikiem decyzyjnym było również zidentyfikowane zapotrzebowanie rynkowe w ramach rodzinnej firmy zajmującej się wynajmem pojazdów. Istniała tam potrzeba

monitorowania eksploatacji floty, jednak dostępne rozwiązania komercyjne okazywały się nieekonomiczne lub zbyt zamknięte na modyfikacje. Budowa własnego rozwiązania pozwoliła uwzględnić aspekt ekonomiczny, oferując funkcjonalną i tańszą alternatywę dostosowaną do specyficznych wymagań użytkownika.

Realizacja pracy wymagała spełnienia szeregu założeń technicznych, w tym implementacji systemu operacyjnego czasu rzeczywistego (FreeRTOS) w celu zapewnienia współbieżności procesów akwizycji i transmisji danych oraz stworzenia obudowy umożliwiającej testy w warunkach drogowych.

Układ pracy został podzielony na rozdziały odpowiadające etapom procesu projektowego. W rozdziale drugim zdefiniowano cel główny oraz cele szczegółowe pracy, a także określono zakres funkcjonalny projektowanego urządzenia. Rozdział trzeci stanowi studium teoretyczne, zawierające przegląd wykorzystanych technologii (m.in. GNSS, MQTT) oraz analizę porównawczą rozwiązań dostępnych na rynku. W rozdziale czwartym przedstawiono proces projektowania i implementacji systemu, obejmujący opis architektury sprzętowej, strukturę oprogramowania oraz integrację z platformą serwerową ThingsBoard. Opisano tu również fizyczne wykonanie prototypu. Rozdział piąty zawiera opis przeprowadzonych testów weryfikacyjnych, w tym testów elektrycznych, funkcjonalnych oraz prób drogowych, wraz z analizą jakości uzyskanych danych. Pracę kończy podsumowanie, zawierające wnioski z realizacji projektu oraz propozycje kierunków dalszego rozwoju systemu.

2. Cel i zakres pracy

Celem głównym jest zaprojektowanie i realizacja urządzenia monitorowania położenia pojazdu z wykorzystaniem mikrokontrolera ESP32.

2.1. Cele szczegółowe pracy:

- Implementację oprogramowania, umożliwiającego akwizycję danych z modułu GNSS, ich archiwizację na karcie pamięci SD oraz zdalną synchronizację z serwerem.
- Zaprojektowanie i wykonanie warstwy sprzętowej urządzenia, w tym obwodu drukowanego (lub układu prototypowego) oraz obudowy.
- Przeprowadzenie testów funkcjonalnych prototypu w warunkach rzeczywistych.
- Analizę jakości pozyskanych danych lokalizacyjnych oraz ocenę skuteczności mechanizmu synchronizacji danych.

2.2. Zakres pracy obejmuje następujące etapy:

- Analiza wymagań funkcjonalnych i przegląd dostępnych rozwiązań technicznych.
- Opracowanie schematu ideowego oraz projektu obwodu drukowanego (PCB).
- Realizacja fizyczna prototypu urządzenia (montaż podzespołów).
- Implementacja oprogramowania sterującego dla układu ESP32.
- Weryfikacja działania urządzenia oraz opracowanie wniosków końcowych.
- Przygotowanie wniosków i propozycji dalszych prac.

2.3. Założenia projektowe

Na podstawie zdefiniowanego celu głównego przyjęto szereg założeń funkcjonalnych, sprzętowych oraz programowych, które determinują sposób realizacji projektu.

2.3.1. Założenia funkcjonalne

- Autonomia działania: Urządzenie powinno działać w sposób bezobsługowy, automatycznie rozpoczynając akwizycję danych po podłączeniu zasilania.
- Rejestracja danych: System musi umożliwiać ciągły zapis parametrów lokalizacyjnych (szerokość i długość geograficzna, czas UTC, prędkość) na lokalnym nośniku danych (karta SD).
- Synchronizacja danych: Urządzenie powinno posiadać mechanizm wykrywania dostępności połączenia sieciowego i automatycznego przesyłania zgromadzonych danych na serwer zewnętrzny (buforowanie danych w trybie offline).
- Sygnalizacja stanu: Zastosowanie diod LED lub innego interfejsu informującego użytkownika o statusie pracy (np. status WiFi, status karty SD, aktywność GNSS).
- Możliwość rozbudowy: Projekt powinien uwzględniać możliwość przyszłego dołączenia interfejsu diagnostycznego pojazdu (OBD/CAN) oraz dodatkowych czujników (np. temperatury).

2.3.2 Założenia sprzętowe

- Jednostka sterująca: Wykorzystanie mikrokontrolera z rodziny ESP32 ze względu na wbudowany moduł komunikacji bezprzewodowej (Wi-Fi/Bluetooth) oraz wysoką wydajność obliczeniową.
- Moduł lokalizacyjny: Zastosowanie modułu GNSS obsługującego standard NMEA, zapewniającego dokładność lokalizacji wystarczającą do śledzenia trasy pojazdu.
- Zasilanie: System powinien być przystosowany do zasilania z instalacji samochodowej (poprzez odpowiedni układ obniżający napięcie) lub zewnętrznego źródła mobilnego (powerbank/akumulator Li-Ion).
- Pamięć masowa: Wykorzystanie karty pamięci SD jako bufora danych o dużej pojemności, umożliwiającego długotrwały zapis historii tras.

- Obudowa: Konstrukcja mechaniczna powinna zapewniać ochronę układów elektronicznych przed uszkodzeniami mechanicznymi oraz umożliwiać łatwy montaż w pojeździe.

2.3.3 Założenia programowe

- Środowisko programistyczne: Oprogramowanie zostanie zrealizowane w języku C++ z wykorzystaniem frameworku Arduino, co zapewni dostęp do szerokiej bazy bibliotek obsługi peryferiów.
- Struktura danych: Dane na karcie SD będą zapisywane w ustrukturyzowanym formacie tekstowym (np. CSV, JSON), co umożliwi ich późniejszą obróbkę i analizę.
- Obsługa błędów: System musi być odporny na typowe błędy, takie jak brak karty SD, utrata sygnału GNSS czy brak połączenia z serwerem, i wznowiać pracę po ustąpieniu awarii.

3. Podstawy teoretyczne i przegląd rozwiązań

Współczesne systemy telemetryczne i lokalizacyjne wymagają integracji wielu warstw technologicznych, począwszy od niskopoziomowej obsługi sprzętu, poprzez systemy operacyjne czasu rzeczywistego, aż po protokoły komunikacyjne warstwy aplikacji. Niniejszy rozdział stanowi szczegółowe studium technologii wybranych do realizacji projektu inżynierskiego. Przedstawiono w nim uzasadnienie doboru poszczególnych rozwiązań w kontekście rygorystycznych wymagań stawianych systemom IoT (Internet of Things) pracującym w środowisku motoryzacyjnym. Analizie poddano paradygmaty komunikacji wewnątrz pojazdowej, standardy transmisji bezprzewodowej oraz metody akwizycji danych lokalizacyjnych.

3.1. Przegląd technologiczny i środowisko programistyczne

Realizacja warstwy programowej (Firmware) urządzenia wbudowanego wymagała wyboru środowiska zapewniającego kompromis między wydajnością a szybkością implementacji. Projekt został zrealizowany w języku C++ z wykorzystaniem frameworka Arduino (biblioteka Arduino.h), co pozwoliło na efektywne wykorzystanie zasobów mikrokontrolera ESP32 oraz dostęp do szerokiej bazy sterowników. Poniższe podrozdziały charakteryzują kluczowe standardy i protokoły, które stanowią fundament działania opracowanego urządzenia.

3.1.1. Magistrala CAN (Controller Area Network)

Magistrala CAN (ang. Controller Area Network) to szeregową magistrala komunikacyjna opracowana przez firmę Robert Bosch GmbH w latach 80. XX wieku, znormalizowana jako ISO 11898. Jest ona obecnie dominującym standardem wymiany danych w systemach motoryzacyjnych oraz automatyce przemysłowej.

CAN wykorzystuje transmisję różnicową, co zapewnia wysoką odporność na zakłócenia elektromagnetyczne - cechę kluczową w środowisku pracy pojazdu. Fizyczne medium transmisyjne stanowi zazwyczaj para skręconych przewodów (skrętka), zakończona rezystorami terminującymi o wartości 120 Ω , co zapobiega odbiciom sygnału. Sygnał interpretowany jest na podstawie różnicy potencjałów między liniami **CAN High** i **CAN Low**:

Stan recesywny (logiczna "1"): Występuje, gdy napięcie na obu liniach jest zbliżone (ok. 2,5 V).

Stan dominujący (logiczne "0"): Występuje, gdy różnica napięć wzrasta (np. CAN High = 3,5 V, CAN Low = 1,5 V).

Sieć CAN działa w topologii magistrali typu multi-master bez jednostki centralnej. Dostęp do łącza regulowany jest metodą **CSMA/CD+CR** (Carrier Sense Multiple Access with Collision Detection and Arbitration on Message Priority). Każda ramka danych posiada unikalny identyfikator (ID), który określa nie tylko zawartość wiadomości, ale także jej priorytet. W przypadku jednoczesnego nadawania przez dwa węzły, wygrywa ten, który nadaje stan dominujący ("0") na bitach identyfikatora o niższej wadze liczbowej (niższy numer ID oznacza wyższy priorytet).

W kontekście diagnostyki pokładowej **OBD-II** (On-Board Diagnostics), wykorzystywanej w projekcie, komunikacja opiera się zazwyczaj na standardzie ISO 15765-4. Umożliwia on odpytywanie sterowników o parametry pracy, takie jak prędkość pojazdu, temperatura płynu chłodzącego czy prędkość obrotowa silnika.

3.1.2. Protokół MQTT (Message Queuing Telemetry Transport)

MQTT to lekki protokół transmisji danych działający w warstwie aplikacji modelu ISO/OSI, wykorzystujący protokół TCP/IP jako warstwę transportową. Został znormalizowany przez OASIS oraz jako standard ISO/IEC 20922. Ze względu na niski narzut danych (nagłówek pakietu może wynosić zaledwie 2 bajty), jest często stosowany w systemach IoT oraz teledystrybucji.

W przeciwieństwie do modelu klient-serwer (znanego np. z HTTP), MQTT wprowadza trzeci podmiot - brokera wiadomości. Architektura składa się z trzech elementów:

Wydawca (Publisher): Urządzenie końcowe (lokalizator), które wysyła dane teledystrybucyjne.

Broker: Serwer pośredniczący, odpowiedzialny za filtrowanie i dystrybucję wiadomości.

Subskrybent (Subscriber): Aplikacja lub baza danych odbierająca informacje.

Taka architektura zapewnia asynchroniczność komunikacji oraz skalowalność systemu - wydawca nie musi znać adresu IP odbiorcy ani jego stanu aktywności. Protokół MQTT definiuje trzy poziomy jakości usług (Quality of Service), które pozwalają dostosować pewność dostarczenia danych do warunków sieciowych (np. niestabilnego połączenia GSM):

QoS 0 (At most once): "Wyślij i zapomnij". Brak potwierdzenia odbioru. Najmniejsze zużycie transferu.

QoS 1 (At least once): Gwarancja dostarczenia (możliwe duplikaty). Wymagane potwierdzenie (PUBACK).

QoS 2 (Exactly once): Gwarancja jednokrotnego dostarczenia. Największy narzut komunikacyjny.

3.1.3. Systemy GNSS i standard NMEA 0183

GNSS (Global Navigation Satellite System) to ogólna nazwa systemów nawigacji satelitarnej, obejmująca m.in. amerykański GPS, europejski Galileo, rosyjski GLONASS oraz chiński BeiDou. Zasada wyznaczania pozycji opiera się na **trilateracji sferycznej**, gdzie odbiornik oblicza swoją lokalizację na podstawie pomiaru czasu propagacji sygnału radiowego z co najmniej czterech satelitów.

Standard NMEA 0183 to dobrowolny standard specyfikacji interfejsu elektrycznego i protokołu danych dla sprzętu morskiego i nawigacyjnego, opracowany przez National Marine Electronics Association. W systemach wbudowanych komunikacja odbywa się zazwyczaj poprzez interfejs asynchroniczny UART. Dane przesyłane są w postaci czytelnych ciągów znaków ASCII, zwanych "zdaniami" (*sentences*). Każda ramka rozpoczyna się od znaku \$ i kończy sekwencją <CR><LF>.

Z punktu widzenia lokalizacji pojazdu, kluczową ramką jest **RMC** (Recommended Minimum Specific GNSS Data). Przykład struktury ramki

\$GPRMC,092750.000,A,5321.6802,N,00630.3372,W,0.02,31.66,280511,,,A*43

zawiera kolejno:

Czas UTC.

Status (A - aktywny/prawidłowy, V - ostrzeżenie/nieprawidłowy).

Szerokość geograficzną (wraz z półkulą N/S).

Długość geograficzną (wraz z półkulą E/W).

Prędkość nad dnem (w węzłach).

Kurs rzeczywisty (w stopniach).

Datę.

Sumę kontrolną (XOR wszystkich znaków między § a *).

Technologia EASY™ (Embedded Assist System) Istotnym rozszerzeniem funkcjonalności w nowoczesnych odbiornikach GNSS (np. opartych na chipsetach MTK) jest technologia EASY™. Służy ona do optymalizacji parametru **TTF** (Time To First Fix). W przeciwieństwie do rozwiązań typu A-GPS wymagających połączenia z siecią, EASY działa autonomicznie - odbiornik oblicza i przewiduje orbity satelitów na okres do 3 dni naprzód, bazując na danych odebranych podczas ostatniej aktywności. Informacje te są przechowywane w podtrzymywanej bateryjnie pamięci RAM lub Flash. Dzięki temu, po ponownym uruchomieniu urządzenia (nawet przy braku bieżącego sygnału z satelitów przez pierwsze sekundy), moduł może błyskawicznie wyznaczyć pozycję, korzystając z zapamiętanych efemeryd.

3.1.4. Magistrala 1-Wire (Interfejs Dallas)

1-Wire to system magistrali komunikacyjnej zaprojektowany przez Dallas Semiconductor (obecnie Analog Devices), który zapewnia stosunkowo niską prędkość transmisji danych na poziomie 16 kbps (lub maksymalnie do 115,2 kbps), sygnalizację oraz zasilanie przez pojedynczy przewód sygnałowy (względem masy).

Magistrala działa w układzie Master-Slave. Kluczową cechą jest wyjście typu Open Drain (otwarty dren) z rezystorem podciągającym (pull-up) do zasilania (zazwyczaj 3,3 V lub 5 V). Umożliwia to realizację tzw. "**zasilania pasożytniczego**" (parasitic power) - urządzenia Slave mogą czerpać energię z linii danych w stanie

wysokim, ładując wewnętrzny kondensator, co pozwala na redukcję okablowania do dwóch przewodów (DATA, GND).

Każde urządzenie 1-Wire posiada fabrycznie nadany, unikalny, 64-bitowy kod ROM, na który składa się:

- 8 bitów kodu rodziny (Family Code): Identyfikuje typ urządzenia (np. termometr).
- 48 bitów unikalnego numeru seryjnego.
- 8 bitów sumy kontrolnej CRC.

W systemach lokalizacji pojazdów interfejs ten jest powszechnie stosowany do:

Autoryzacji kierowców: Wykorzystanie pastylek iButton (Dallas Key), które kierowca przykładą do czytnika w celu identyfikacji przed rozpoczęciem jazdy.

Monitoringu temperatury: Wykorzystanie cyfrowych czujników (np. DS18B20) w przestrzeni ładunkowej pojazdów chłodni.

3.2. Analiza wybranych rozwiązań rynkowych

Rynek systemów lokalizacji pojazdów jest obecnie nasycony rozwiązaniami o zróżnicowanej funkcjonalności, cenie oraz przeznaczeniu. Od prostych rejestratorów tras, przez systemy antykradzieżowe, aż po zaawansowaną telematykę flotową. W celu osadzenia realizowanego projektu w kontekście rynkowym, dokonano analizy trzech reprezentatywnych urządzeń dostępnych w sprzedaży oraz porównano je z projektowanym rozwiązaniem opartym na mikrokontrolerze ESP32.

Do przeglądu wybrano urządzenia reprezentujące różne segmenty rynku:

- klasyczny lokalizator montowany na stałe,
- rozwiązanie typu „Plug&Play” pod złącze OBD;
- przenośny lokalizator z własnym zasilaniem.

3.2.1. Lokalizator GPS BearPoint C18



Rys. 3.1. Urządzenie BearPoint C18 <https://bearpoint.pl/produkt/lokalizator-gps-bearpoint/>

BearPoint C18 stanowi przykład klasycznego lokalizatora montowanego na stałe w instalacji elektrycznej pojazdu, dedykowanego zarówno klientom indywidualnym, jak i zarządom flot. Warstwa komunikacyjna urządzenia bazuje na technologii 4G/LTE z mechanizmem automatycznego przełączania do sieci 2G, co gwarantuje stabilność zasięgu. Konstrukcja sprzętowa uwzględnia wbudowaną baterię, podtrzymującą pracę modułu w przypadku awarii lub sabotażu zasilania głównego. W zakresie funkcjonalności system oferuje śledzenie pozycji w czasie rzeczywistym, obsługę geostref, alerty o przekroczeniu prędkości oraz możliwość zdalnego unieruchomienia pojazdu poprzez wysterowanie przekaźnika zapłonu. Z perspektywy modelu biznesowego rozwiązanie to opiera się na udostępnieniu przez producenta gotowego środowiska wizualizacyjnego, co zazwyczaj wiąże się z modelem abonamentowym i generuje stałe koszty eksploatacji.

3.2.2. Lokalizator GPS OBD MK08



Rys. 2.2 Urządzenie MK08 <https://szpiegujemy.com/product-pol-1690>

MK08 to przedstawiciel kategorii urządzeń „Plug & Play”, które wykorzystują złącze diagnostyczne OBDII jako źródło zasilania oraz punkt montażowy. Kluczową zaletą tego rozwiązania jest brak konieczności ingerencji w instalację elektryczną pojazdu, proces instalacji ogranicza się do wpięcia urządzenia w gniazdo diagnostyczne. W zakresie funkcjonalności moduł oferuje podstawowe usługi lokalizacyjne, archiwizację historii tras oraz system alarmów wstrząsowych. Warto jednak odnotować, że budżetowe wersje tego typu rozwiązań często wykorzystują interfejs OBD wyłącznie do celów zasilających, nie implementując pełnej obsługi protokołu diagnostycznego (brak odczytu parametrów takich jak RPM czy kody błędów), co stanowi istotne ograniczenie względem zaawansowanych systemów telematycznych. Ciągłe zasilanie z instalacji pojazdu rozwiązuje problem konieczności ładowania baterii wewnętrznej, lecz w przypadku dłuższego postoju może prowadzić do nadmiernego obciążenia akumulatora rozruchowego.

3.2.3. Lokalizator GPS MK6B 4G (Magnetyczny)



Rys. 2.3. Urządzenie MK6B <https://szpiegujemy.com/product-pol-1696>

Model MK6B reprezentuje segment lokalizatorów mobilnych, działających niezależnie od stałej instalacji elektrycznej pojazdu. Cechą wyróżniającą to rozwiązanie jest implementacja ogniwa o wysokiej pojemności (3000 mAh), co pozwala na osiągnięcie czasu pracy wynoszącego nawet 35 dni, w zależności od skonfigurowanej częstotliwości próbkowania i wysyłania danych. Urządzenie wyposażono w zintegrowany montaż magnetyczny, umożliwiający szybką instalację na zewnętrznych elementach karoserii lub kontenerach transportowych. Warstwa komunikacyjna oparta jest na modemie 4G LTE, zapewniającym transmisję danych w czasie rzeczywistym. Ze względu na swoją autonomiczność energetyczną, MK6B znajduje zastosowanie głównie w monitoringu doraźnym, zabezpieczaniu ładunków oraz śledzeniu maszyn budowlanych pozbawionych dostępu do pokładowej sieci zasilającej.

3.2.4. Kategoryzacja dostępnych rozwiązań

Na podstawie przeglądu rynku można wyróżnić trzy główne kategorie systemów lokalizacji, różniące się architekturą i grupą docelową:

1. **Gotowe komercyjne trackery GPS (np. BearPoint, MK08, MK6B):** Są to kompletne produkty, często zamknięte (proprietary), zintegrowane z kartą SIM i dedykowaną chmurą producenta. Zapewniają wysoką niezawodność i łatwość obsługi, ale wiążą się z kosztami abonamentowymi i brakiem możliwości modyfikacji oprogramowania czy sposobu przetwarzania danych.
2. **Rozwiązania OEM (Original Equipment Manufacturer):** Systemy fabrycznie montowane w nowoczesnych pojazdach, głęboko zintegrowane z magistralą CAN i systemami multimedialnymi. Oferują najwyższą jakość danych diagnostycznych, ale są zamknięte dla użytkownika i trudne do przeniesienia do starszych pojazdów.
3. **Modułowe rozwiązania DIY (Projekt inżynierski):** Systemy oparte na uniwersalnych mikrokontrolerach (ESP32, Arduino) i oddzielnych modułach GNSS/GSM. Cechują się wysoką elastycznością, niskim kosztem początkowym i pełną kontrolą nad danymi, wymagają jednak samodzielnej integracji sprzętowej i programowej.

3.2.5. Analiza porównawcza

Poniższa tabela przedstawia porównanie analizowanych rozwiązań komercyjnych z prototypem realizowanym w ramach niniejszej pracy inżynierskiej.

Tabela 3.1. Porównanie rozwiązań

Kryterium	Komercyjne	OEM	DIY
Dokładność lokalizacji	Wysoka	Bardzo wysoka	Zależna od modułu
Ciągłość przesyłu	Wysoka / Ciągła	Wysoka / Ciągła	Zależna
Zasilanie	Złącze pojazdu (OBD) + bateria	Instalacja pojazdu	Bateryjne (Li-Ion 18650) + możliwość OBD
Koszt	Średni+ Abonament	Wysoki	Niski
Otwartość systemu	Zamknięty	Całkowicie zamknięty	Otwarty

3.2.6. Wnioski z przeglądu i uzasadnienie wyboru technologii

Analiza konkurencyjnych rozwiązań prowadzi do następujących wniosków, które ukształtowały założenia projektowe niniejszej pracy:

- **Zasadność podejścia modułowego:** Dla celów edukacyjnych i prototypowych, wykorzystanie platformy ESP32 jest optymalne. Pozwala na drastyczne obniżenie kosztów budowy urządzenia oraz umożliwia szybkie iteracje (zmiany w kodzie i sprzęcie), co jest niemożliwe w zamkniętych produktach komercyjnych.
- **Akceptacja ograniczeń komunikacyjnych:** Wybór Wi-Fi jako głównego kanału transmisji jest świadomym kompromisem. Choć komercyjne trackery (MK6B, BearPoint) oferują ciągły monitoring przez LTE, w zastosowaniach takich jak analiza tras po fakcie, logistyka wewnętrzna czy "czarne skrzynki", synchronizacja okresowa (Batch Upload) po powrocie do zasięgu znanej sieci jest wystarczająca i tańsza w eksploatacji (brak karty SIM).
- **Potencjał integracji OBD2:** Przegląd urządzenia MK08 wskazuje na duży potencjał złącza diagnostycznego. Jednakże, aby w projekcie DIY uzyskać funkcjonalność wykraczającą poza samo zasilanie, konieczne byłoby zastosowanie dodatkowego transceivera CAN (np. TJA1050) oraz implementacja stosu protokołów ISO 15765-4 / SAE J1979.

Podsumowując, realizowany projekt nie stanowi bezpośredniej konkurencji dla systemów antykradzieżowych (wymagających LTE), lecz jest elastyczną platformą telemetryczną, oferującą funkcjonalności niedostępne w tanich trackerach, takie jak pełny dostęp do surowych danych i możliwość dowolnej konfiguracji parametrów zapisu.

4. Projekt i implementacja urządzenia

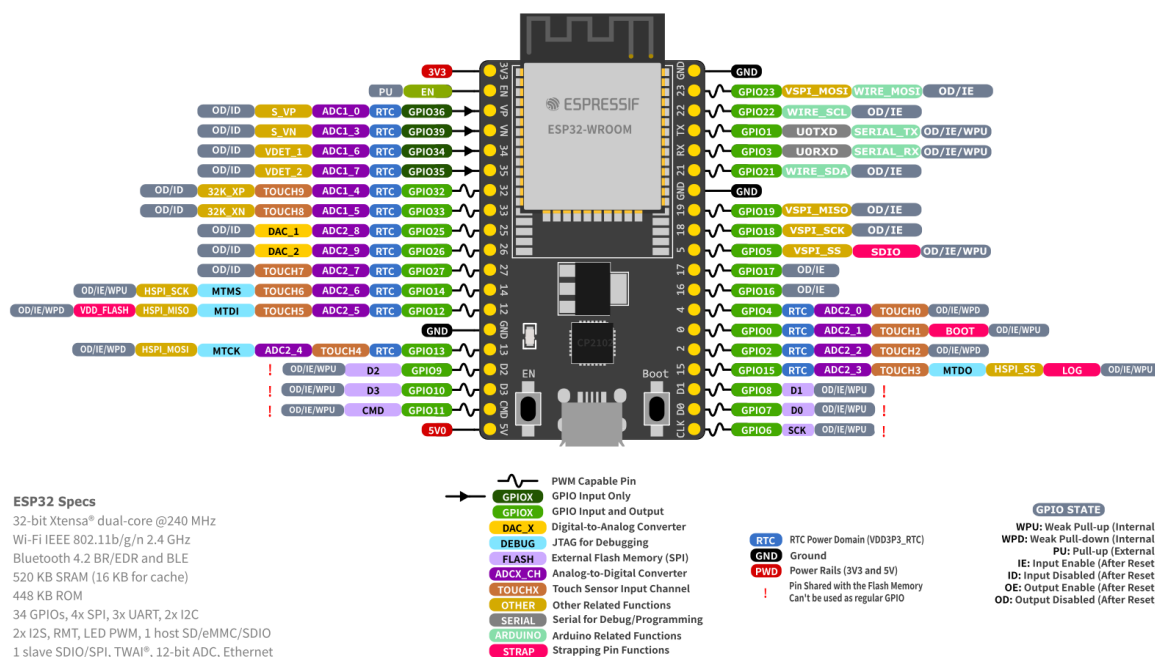
Rozdział ten opisuje szczegóły projektowe warstwy sprzętowej i programowej, prezentując sposób, w jaki omówione wcześniej technologie zostały zintegrowane w spójny system.

4.1. Architektura sprzętowa

Zaprojektowany system akwizycji danych opiera się na architekturze modułowej, w której jednostka centralna zarządza peryferiami odpowiedzialnymi za pozycjonowanie, komunikację z magistralą pojazdu, pomiary środowiskowe oraz archiwizację danych. Poniżej przedstawiono szczegółową charakterystykę poszczególnych bloków funkcjonalnych urządzenia.

4.1.1. Jednostka centralna

Jako główny sterownik systemu wybrano moduł deweloperski ESP32-DevKitC-32D v4 firmy Espressif Systems. Decyzja ta podyktowana była obecnością dwurdzeniowego procesora, który umożliwia jednoczesną obsługę stosu komunikacyjnego Wi-Fi oraz zadań czasu rzeczywistego (akwizycja danych z czujników). Mikrokontroler pracuje w logice 3.3 V i dysponuje wystarczającą liczbą interfejsów sprzętowych (UART, SPI, TWAI/CAN) do obsługi wszystkich założonych modułów bez konieczności stosowania ekspanderów wyprowadzeń.



Rys. 4.1. Schemat wyprowadzeń ESP32-DevKitC V4 https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32/esp32-devkitc/user_guide.html

4.1.2. Moduł pozycjonowania GNSS

Do określania pozycji geograficznej wykorzystano układ Quectel LC76G. Jest to odbiornik Multi-GNSS obsługujący systemy GPS, GLONASS, Galileo oraz BeiDou, co zapewnia wysoką precyzję i szybkość ustalania pozycji nawet w trudnych warunkach terenowych. Komunikacja z mikrokontrolerem odbywa się za pośrednictwem interfejsu UART (piny GPIO 16 i 17), przy prędkości transmisji 115200. Dodatkowo, system wykorzystuje sygnał PPS (Pulse Per Second) podłączony do pinu GPIO 32 w celu precyzyjnej synchronizacji czasu systemowego.

4.1.3. Komunikacja z magistralą CAN

W celu odczytu danych telemetrycznych z pojazdu (np. prędkość obrotowa silnika, prędkość pojazdu), zastosowano zewnętrzny transceiver CAN-PAL oparty na układzie TJA1051T/3. Pełni on rolę medium fizycznego, konwertując poziomy napięcie magistrali różnicowej CAN na poziomy logiczne akceptowane przez sterownik CAN (TWAI)

wbudowany w układ ESP32. Transceiver podłączono do pinów GPIO 21 (RX) oraz GPIO 22 (TX).

4.1.4 Pomiar temperatury

Do pomiaru temperatury wykorzystano cyfrowy czujnik temperatury DS18B20, wykorzystujący magistralę 1-Wire, który został podpięty do GPIO 4 co pozwoliło na monitorowanie temperatury otoczenia lub, w zależności od montażu, temperatury wewnątrz pojazdu.

4.1.5. Pamięć masowa

W celu zapewnienia redundancji danych oraz możliwości pracy w trybie offline, system wyposażono w moduł czytnika kart microSD komunikujący się poprzez magistralę SPI (piny GPIO 5, 18, 19, 23). Umożliwia to buforowanie danych do archiwizacji lub w przypadku braku połączenia z siecią Wi-Fi.

4.1.6. Układ zasilania

Ze względu na specyfikę pracy w pojeździe, zaprojektowano hybrydowy układ zasilania:

- Wejścia zasilania: Urządzenie może być zasilane z instalacji 12V pojazdu (poprzez złącze OBDII) lub z portu USB-C.
- Przetwornica Step-Down: Napięcie 12V jest obniżane do 5V za pomocą przetwornicy impulsowej.
- Zabezpieczenie przetwornicy: Zastosowano diodę Schottky’ego co pozwala na bezprzerwowe przełączanie między zasilaniem USB-C lub 12V z instalacji pojazdu.
- Zasilanie awaryjne (UPS): Ogniwo Li-Ion typu 18650 (Panasonic NCR18650B) wraz z modułem ładowania i przetwornicą podwyższającą (Boost) zapewnia ciągłość pracy urządzenia po wyłączeniu zapłonu w pojeździe.

4.2. Architektura oprogramowania

Oprogramowanie urządzenia zostało zaimplementowane w języku C++ z wykorzystaniem frameworka Arduino dla platformy ESP32. Ze względu na konieczność równoległej obsługi wielu procesów asynchronicznych, takich jak akwizycja danych z czujników, obsługa stosu sieciowego TCP/IP, operacje plikowe oraz obsługa żądań HTTP, zrezygnowano z klasycznej pętli głównej (super-loop) na rzecz systemu operacyjnego czasu rzeczywistego FreeRTOS.

4.2.1. Organizacja kodu źródłowego i struktura plików

Kod źródłowy projektu został przygotowany w środowisku Visual Studio Code z wykorzystaniem Arduino IDE jako narzędzia do wgrywania kodu na ESP32. Zarządzanie cyklem życia oprogramowania oraz historią zmian realizowano przy użyciu rozproszonego systemu kontroli wersji Git. Kompletne repozytorium projektu, zawierające pełną strukturę katalogów, pliki konfiguracyjne oraz dokumentację techniczną, zostało umieszczone w serwisie hostingowym GitHub. Takie podejście umożliwia łatwą replikację środowiska programistycznego oraz weryfikację poszczególnych etapów powstawania oprogramowania. Kod źródłowy został podzielony na moduły funkcjonalne zgodnie z paradygmatem programowania obiektowego. Poniżej przedstawiono charakterystykę kluczowych plików projektu:

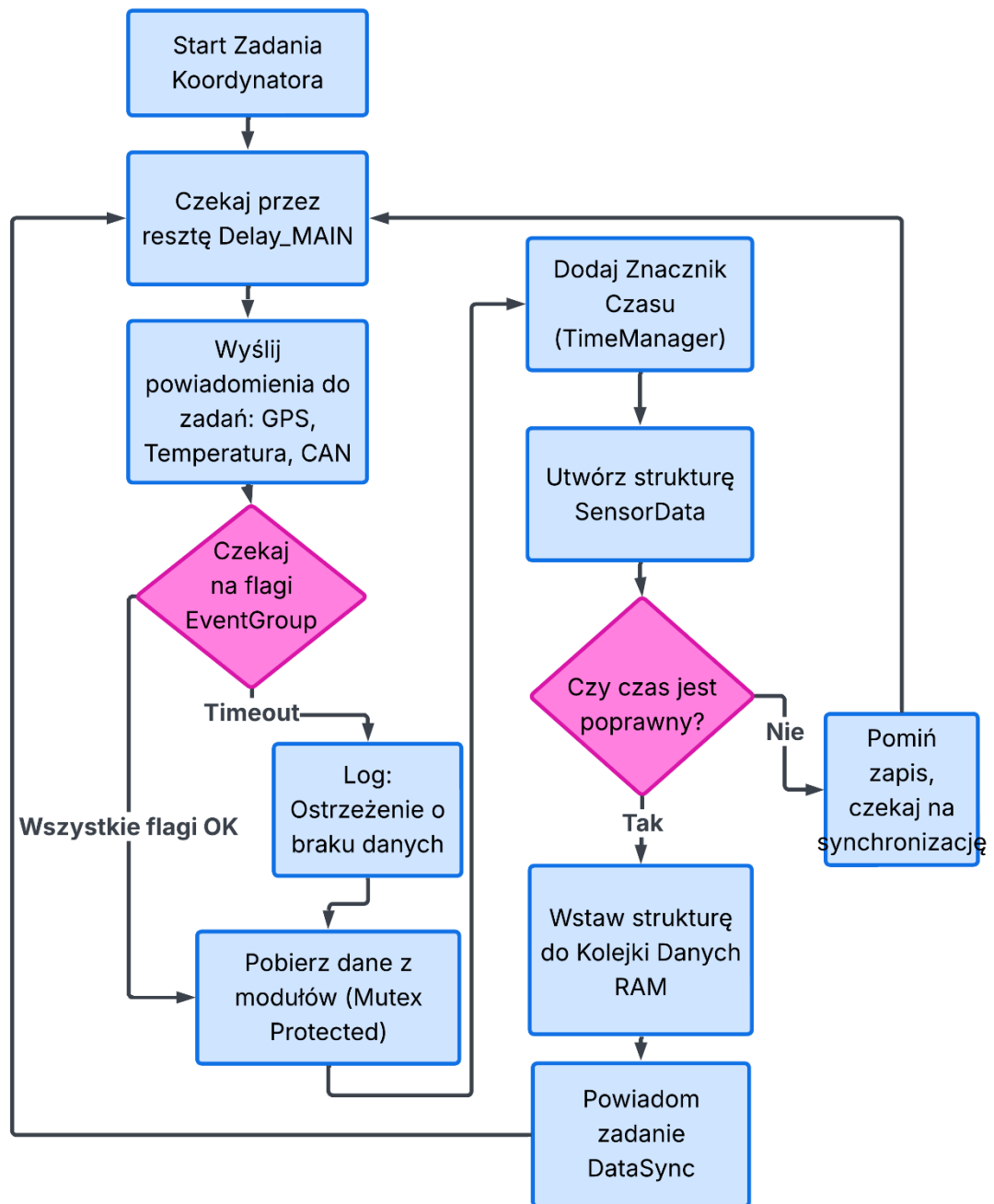
- `main.ino`: Zawiera konfigurację sprzętową (`setup()`), inicjalizację obiektów systemowych (kolejki, semaforey) oraz implementację głównego zadania `CoordinatorTask`.
- `SensorData.h`: Plik nagłówkowy definiujący globalną strukturę danych oraz mapowania dla serializacji JSON (X-Macro). Jest to jedyne miejsce definicji parametrów telemetrycznych, co gwarantuje spójność w całym projekcie.
- `SdModule`: Zaawansowany menedżer pamięci masowej. Odpowiada za fizyczny zapis na kartę SD, rotację plików archiwalnych.
- `ThingsBoardClient`: Obsługuje protokół MQTT dla platformy Thingsboard, w tym autoryzację, przesyłanie telemetry oraz odbiór atrybutów zdalnej konfiguracji.

- WiFiManager: Moduł zarządzający łącznością. Implementuje logikę skanowania i wyboru najlepszego punktu dostępowego z listy znanych sieci.
- WebServerModule: Obsługuje lokalny serwer HTTP, udostępniając interfejs mapy.
- GpsModule / CanModule: Klasy warstwy abstrakcji sprzętowej. Ukrywają one złożoność komunikacji UART/TWAI, udostępniając wyższym warstwom gotowe, przetworzone dane (np. prędkość w km/h, współrzędne geograficzne).
- TimeManager: Moduł odpowiedzialny za synchronizację czasu. Zarządza priorytetami źródeł czasu (GPS > NTP > RTC) i dostarcza spójny znacznik czasowy Unix Timestamp dla wszystkich logowanych zdarzeń.

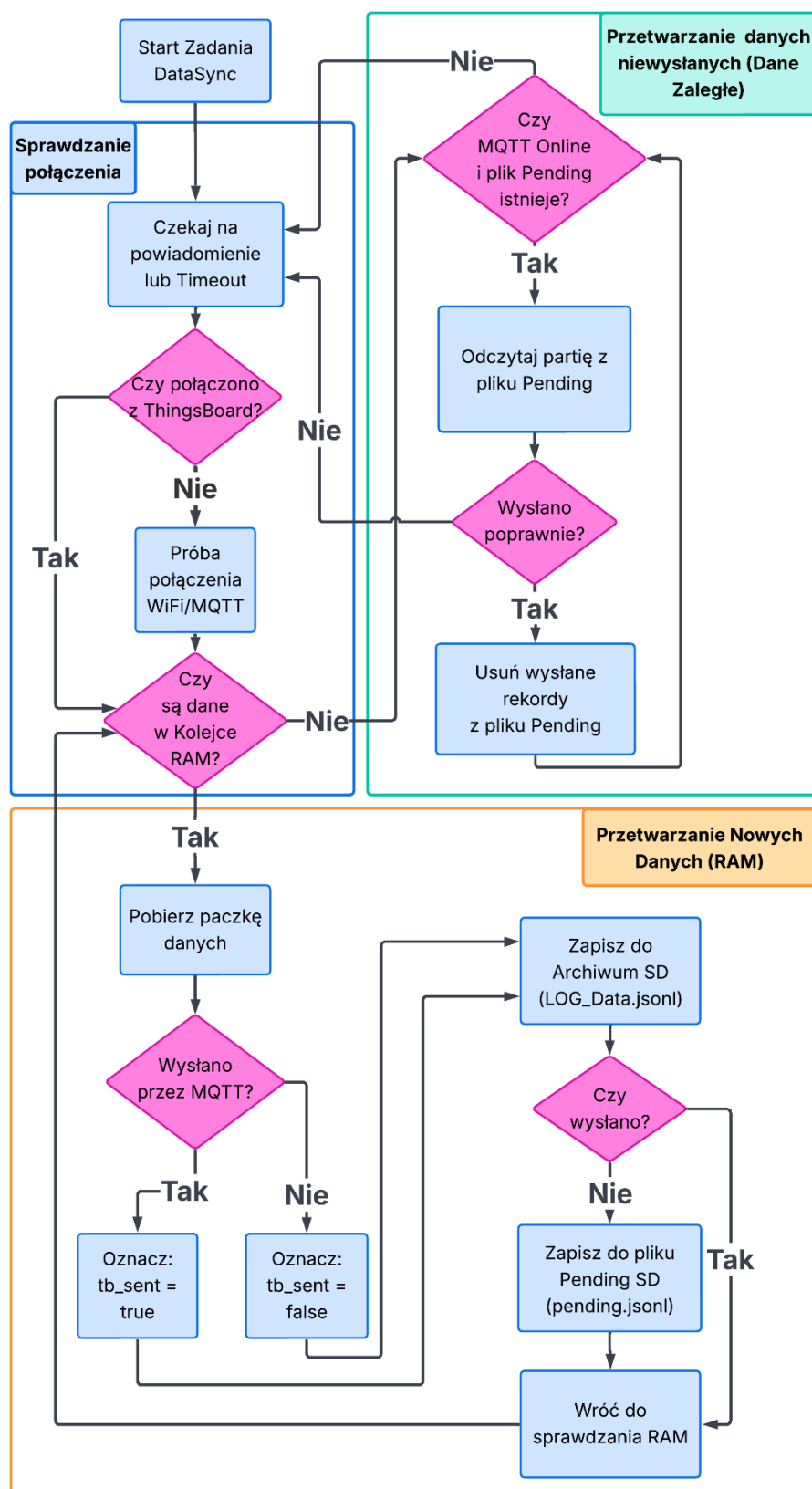
4.2.2. Podział na zadania (Tasks)

System operacyjny zarządza pulą niezależnych wątków o zróżnicowanych priorytetach:

- CoordinatorTask: Cykliczne wyzwalanie pomiarów i agregacja danych (Snapshot).
- TaskGPS / TaskTemp / TaskCAN: Obsługa poszczególnych czujników i protokołów komunikacyjnych.
- TaskDataSync: Krytyczny proces odpowiedzialny za opróżnianie bufora danych (wysyłka MQTT oraz zapis SD).
- TaskWiFi: Monitorowanie stanu połączenia i automatyczna reasocjacja.
- TaskWebServer: Obsługa żądań HTTP od klientów lokalnych (przeglądarka).
- TaskSleep: Zarządzanie energią i procesem bezpiecznego wyłączania urządzenia.



Rys. 4.2 Diagram funkcjonalny zadania koordynatora



Rys. 4.3 Diagram funkcjonalny zadania synchronizacji danych

4.2.3. Użyte biblioteki

Efektywna realizacja oprogramowania wbudowanego wymagała wykorzystania sprawdzonych bibliotek open-source, które zapewniają stabilność działania oraz abstrakcję warstwy sprzętowej. Poniżej zestawiono kluczowe biblioteki użyte w projekcie wraz z uzasadnieniem ich doboru:

- **ArduinoJson** - jest to de facto standard w ekosystemie Arduino służący do serializacji i deserializacji danych w formacie JSON. W projekcie biblioteka ta pełni kluczową rolę w formowaniu pakietów telemetrycznych wysyłanych do brokera MQTT. Pozwala na dynamiczne tworzenie struktur danych, co jest niezbędne przy zmiennej liczbie parametrów.
- **PubSubClient** - lekka biblioteka implementująca klienta protokołu MQTT. Została wybrana ze względu na swoją niezawodność, minimalne zużycie pamięci oraz pełną obsługę poziomów Quality of Service. Odpowiada za utrzymywanie połączenia z brokerem, obsługę mechanizmu Keep-Alive oraz publikowanie i subskrybowanie tematów.
- **TinyGPS++** - biblioteka służąca do parsowania surowego strumienia danych NMEA 0183, odbieranego z modułu GNSS przez interfejs UART. Jej obiektowa struktura pozwala na łatwą ekstrakcję kluczowych informacji, takich jak szerokość i długość geograficzna, wysokość n.p.m., prędkość oraz precyzyjny czas, eliminując konieczność ręcznego analizowania ciągów znaków.
- **OneWire** oraz **DallasTemperature** - zestaw bibliotek obsługujący komunikację na magistrali 1-Wire. OneWire realizuje niskopoziomową obsługę protokołu, natomiast DallasTemperature dostarcza wysokopoziomowy interfejs do obsługi cyfrowych czujników temperatury z rodziny DS18B20, umożliwiając ich adresowanie i odczyt w trybie pasożytniczym.
- **ESP32-TWAI-CAN** - biblioteka stanowiąca wrapper dla wbudowanego w układ ESP32 kontrolera TWAI (Two-Wire Automotive Interface), kompatybilnego ze standardem CAN 2.0B. Umożliwia ona filtrowanie ramek sprzętowych oraz obsługę przerwań, co jest kluczowe dla płynnego odczytu danych z magistrali pojazdu bez blokowania głównego wątku procesora.

4.2.4. Struktura danych i technika X-Macro

W celu zapewnienia ścisłej spójności między strukturą danych w pamięci RAM mikrokontrolera a formatami wyjściowymi (JSON), w pliku nagłówkowym `SensorData.h` zastosowano zaawansowaną technikę preprocesora C/C++ zwaną **X-Macro**. Pozwala ona na zdefiniowanie listy pól danych w jednym centralnym miejscu, co eliminuje redundancję kodu i zmniejsza ryzyko błędów.

Poniżej przedstawiono fragment definicji makra mapującego:

```
#define SENSOR_DATA_MAP(XX) \
    XX(uint64_t, ts, "ts", false, true) \
    XX(double, lat, "lat", true, true) \
    /* ... kolejne definicje pól ... */
```

Makro to jest rozwijane dwukrotnie w procesie kompilacji, pełniąc dwie odrębne funkcje:

1. Do budowy struktury `struct SensorData` w pamięci urządzenia.
2. Do generowania funkcji `sensorDataToTb` oraz `sensorDataToSd`, które mapują pola struktury na odpowiednie klucze JSON.

Zastosowanie tej techniki eliminuje ryzyko powszechnych błędów programistycznych, takich jak pominięcie nowo dodanego pola w procedurze serializacji, co jest zgodne z zasadą **DRY** (Don't Repeat Yourself).

4.2.4. Formaty danych

System wykorzystuje format JSON, jednak jego struktura różni się w zależności od medium docelowego:

- Format chmurowy (ThingsBoard MQTT): Dane pomiarowe zgrupowane są w obiekcie values, co jest wymagane przez API platformy:

```
[
  {
    "ts": 1767307224811,
    "values": {
      "ts_source": 2,
      "lat": 50.0691678,
      "lon": 19.9044705,
      "alt": 220.32,
      "vel": 0.87044,
      "temp": 18.8125,
      "can_vel": 0,
      "ec": 0,
      "rssi": -36
    }
  }, {
    "ts": 1767307224811,
    "values": {
      "ts_source": 2,
      "lat": 50.0691678,
      "lon": 19.9044705,
      "alt": 220.32,
      "vel": 0.87044,
      "temp": 18.8125,
      "can_vel": 0,
      "ec": 0,
      "rssi": -36
    }
  }
]
```

- Format lokalny (Karta SD): Stosowany jest format JSON Lines, gdzie każdy wiersz stanowi płaski obiekt JSON:

```
{
  "ts":1767307224811,
  "ts_source":2,
  "lat":50.0691678,
  "lon":19.9044705,
  "alt":220.32,
  "vel":0.87044,
  "temp":18.8125,
  "can_vel":0,
  "lgr_ts":1767307224044,
  "ltr_ts":1767307224811,
  "lcr_ts":0,
  "ec":0,
  "tb_sent":true,
  "rssi":-36
}
```

4.2.5. Zaawansowana obsługa pamięci (SdModule)

Moduł SdModule implementuje mechanizmy bezpieczeństwa i organizacji danych:

1. Automatyczna rotacja archiwum: Dane historyczne zapisywane są w plikach z datą w nazwie. Po przekroczeniu 5 MB tworzony jest nowy plik, co zapobiega fragmentacji systemu plików.
2. Buforowanie Offline (FIFO): W przypadku braku sieci, dane trafiają do pliku pending.jsonl. Po odzyskaniu połączenia, system wysyła starsze rekordy, a następnie usuwa je z pliku, gwarantując chronologię zdarzeń bez duplikacji danych.
3. Sprawdzanie karty: Przed każdym zapisem weryfikowana jest obecność karty SD. W razie błędu moduł podejmuje próbę ponownej inicjalizacji magistrali SPI.

4.2.6. Mechanizmy synchronizacji i komunikacji międzyprocesowej

W celu zapewnienia integralności danych współdzielonych pomiędzy wątkami zastosowano mechanizmy synchronizacji dostępne w FreeRTOS:

- Grupy zdarzeń (Event Groups): Wykorzystywane przez Koordynatora do oczekiwania na zakończenie pracy przez wszystkie aktywne czujniki (EVENT_GPS_READY, EVENT_TEMP_READY, EVENT_CAN_READY). Pozwala to na synchronizację pomiarów, mimo różnego czasu ich trwania dla poszczególnych sensorów.
- Kolejki (Queues): Do przekazywania danych między Koordynatorem a zadaniem synchronizacji (TaskDataSync) zastosowano kolejkę dataQueue działającą jako bufor kołowy. Zapobiega to utracie próbek w sytuacjach, gdy zapis na kartę SD lub wysyłka sieciowa ulegną chwilowemu opóźnieniu.
- Semaforey (Mutexes): Zabezpieczają dostęp do współdzielonych zasobów sprzętowych, takich jak magistrala SPI (dla karty SD) oraz struktura danych pomiarowych, eliminując ryzyko wyścigów (Race Conditions).

4.2.7. Zarządzanie danymi i tryb Offline

System implementuje hybrydowy model zapisu danych, zapewniający ciągłość historii pomiarowej niezależnie od stanu połączenia sieciowego:

- Archiwizacja ciągła: Każdy rekord pomiarowy jest bezwarunkowo zapisywany w lokalnym archiwum na karcie SD w formacie JSON Lines (.jsonl). Pliki są rotowane automatycznie na podstawie rozmiaru.
- Kolejowanie Offline (Pending): W przypadku braku połączenia z serwerem MQTT, dane trafiają dodatkowo do pliku buforowego (pending.jsonl). Po odzyskaniu połączenia, zadanie TaskDataSync wysyła zaległe rekordy, a następnie usuwa je z bufora lokalnego.

4.2.8. Synchronizacja czasu

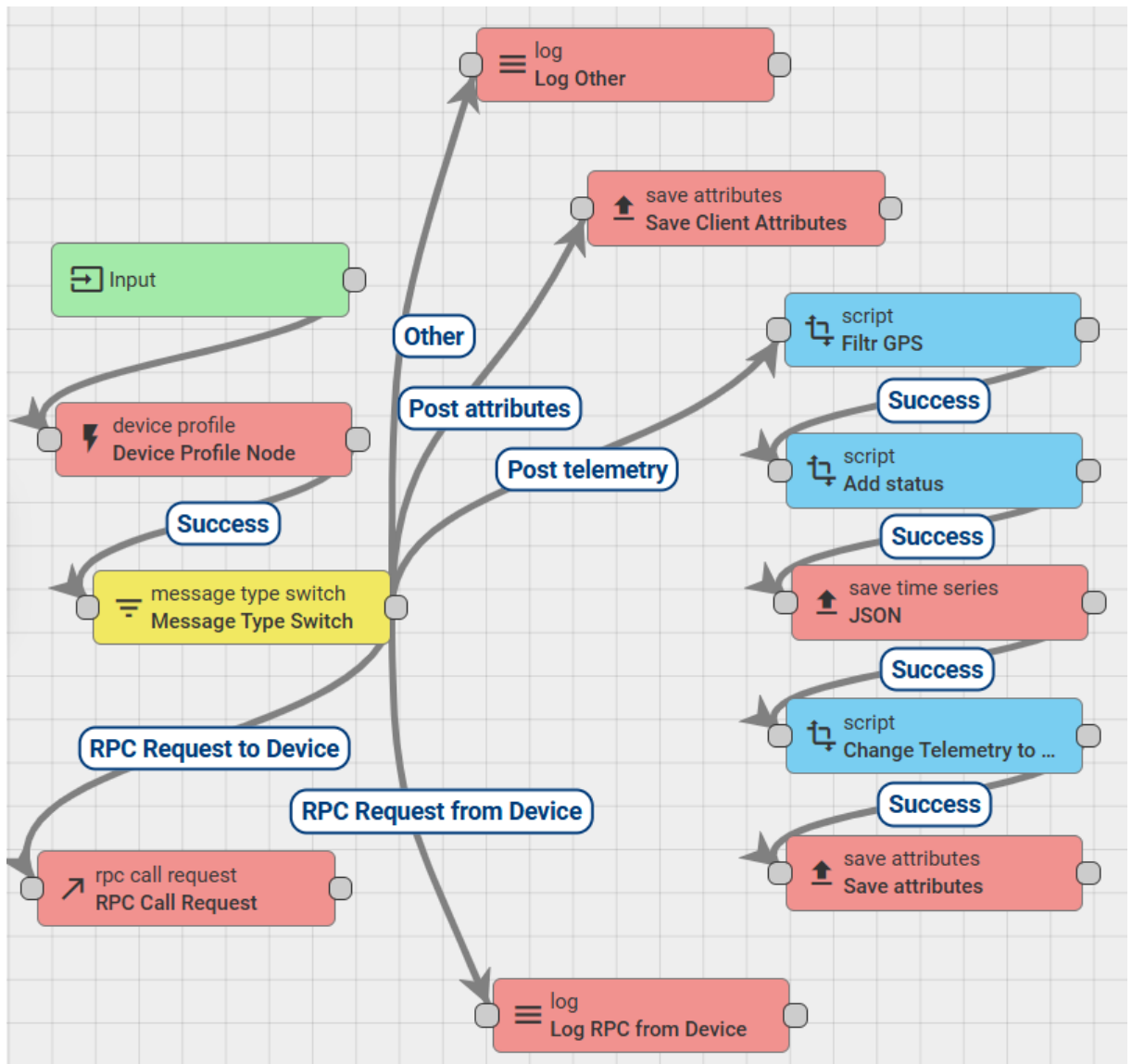
Kluczowym elementem systemu jest moduł TimeManager, który utrzymuje spójny czas systemowy (Unix Timestamp). System priorytetowo synchronizuje się z precyzyjnym sygnałem GPS, a w przypadku jego braku płynnie przechodzi na czas pobrany z sieci Wi-Fi (jeśli dostępny) lub wewnętrzny licznik RTC, gwarantując chronologiczną spójność logowanych danych.

4.3. Integracja z platformą ThingsBoard

Integralną częścią systemu jest warstwa serwerowa, zrealizowana w oparciu o platformę IoT ThingsBoard. Pełni ona rolę brokera MQTT, silnika reguł (Rule Engine) oraz interfejsu użytkownika (Dashboard). Rozwiązanie to pozwala na centralizację danych pomiarowych, ich wizualizację w czasie rzeczywistym oraz zdalne zarządzanie parametrami pracy urządzenia bez konieczności fizycznej ingerencji w oprogramowanie układowe.

4.3.1. Przetwarzanie danych i Silnik Reguł

Logika przetwarzania przychodzących komunikatów MQTT została zdefiniowana za pomocą graficznego edytora reguł (Rule Chain). Strumień danych z urządzenia trafia do węzła wejściowego, gdzie następuje jego klasyfikacja.

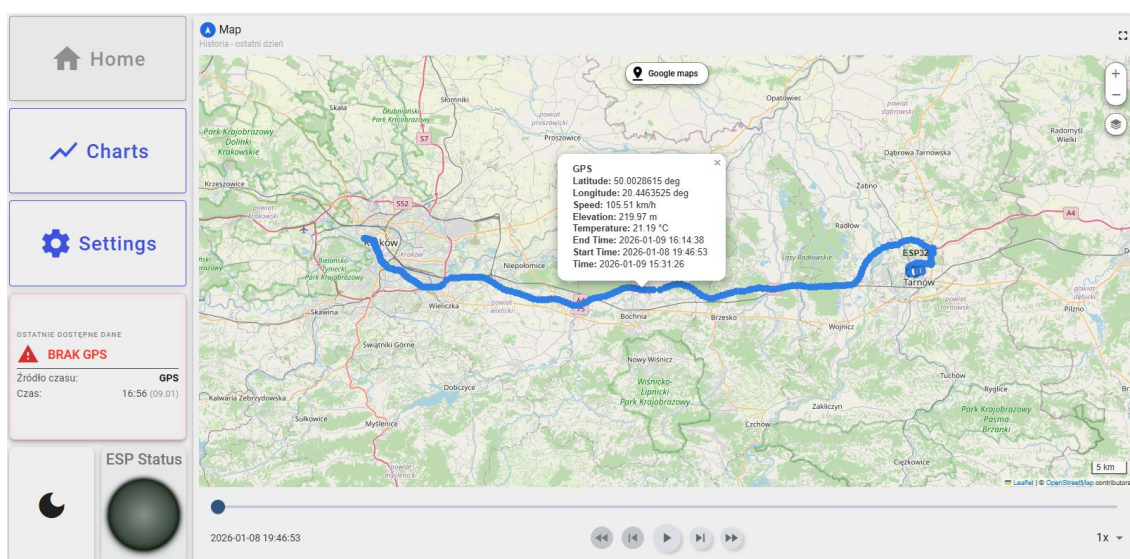


Rys. 4.4 Schemat łańcucha reguł (Rule Chain) skonfigurowanego na serwerze. Widoczne węzły odpowiedzialne za filtrację danych GPS, zapis telemetrii oraz aktualizację atrybutów.

Jak przedstawiono na Rysunku 4, system automatycznie rozdziela komunikaty na telemetrię (dane pomiarowe) oraz atrybuty (statusy). Zastosowano dedykowane skrypty przetwarzające (Nodes), m.in. Filtr GPS, który wstępnie weryfikuje poprawność współrzędnych przed ich zapisem do bazy danych, oraz skrypty aktualizujące status aktywności urządzenia.

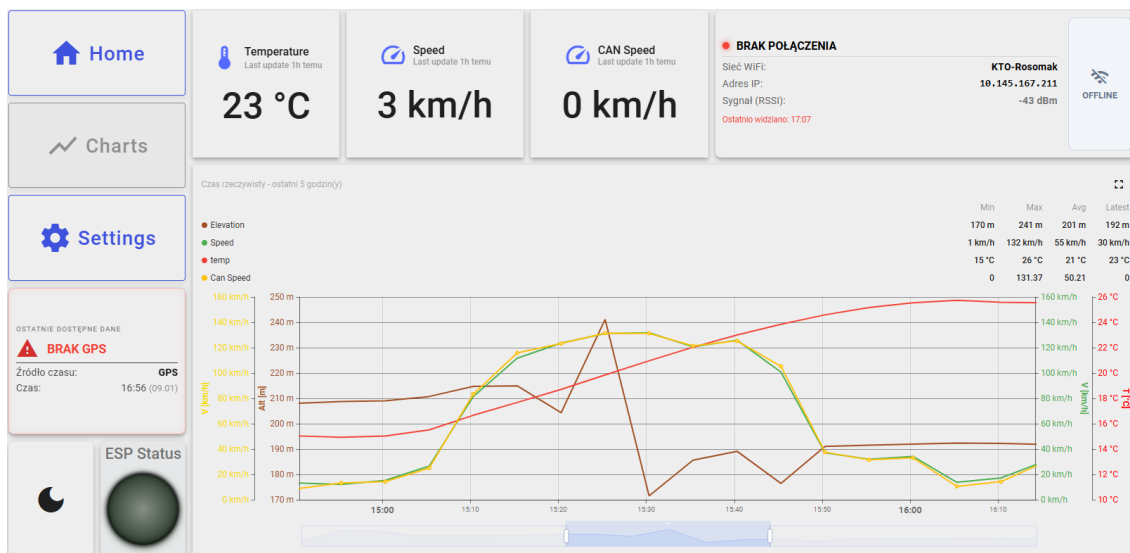
4.3.2. Wizualizacja danych i monitoring

Interfejs użytkownika został zaprojektowany w formie interaktywnego pulpitu (Dashboard), podzielonego na sekcje tematyczne. Sekcja mapy (Rys. 5) wykorzystuje widget "Route Map", który na podkładzie OpenStreetMap wizualizuje aktualną pozycję pojazdu oraz historię przebytej trasy. Interfejs umożliwia odtwarzanie przebiegu trasy w czasie (funkcja Time Window), co pozwala na analizę dynamiki przemieszczania się obiektu.



Rys. 4.5. Wizualizacja trasy przejazdu na mapie cyfrowej. Widoczny znacznik ostatniej pozycji oraz ślad historii przemieszczania.

Do monitorowania parametrów bieżących przygotowano panel "Charts" (Rys. 6). Zawiera on wskaźniki analogowe (prędkościomierz) oraz cyfrowe, prezentujące temperaturę otoczenia, poziom sygnału Wi-Fi (RSSI) oraz status połączenia. Wykresy liniowe u dołu ekranu pozwalają na korelację parametrów w czasie, np. wpływu wzniesień terenu (Elevation) na prędkość pojazdu.



Rys. 4.6. Panel telemetryczny prezentujący parametry bieżące (prędkość, temperatura) oraz status diagnostyczny połączenia sieciowego.

4.3.3. Dwukierunkowa komunikacja i zdalna konfiguracja

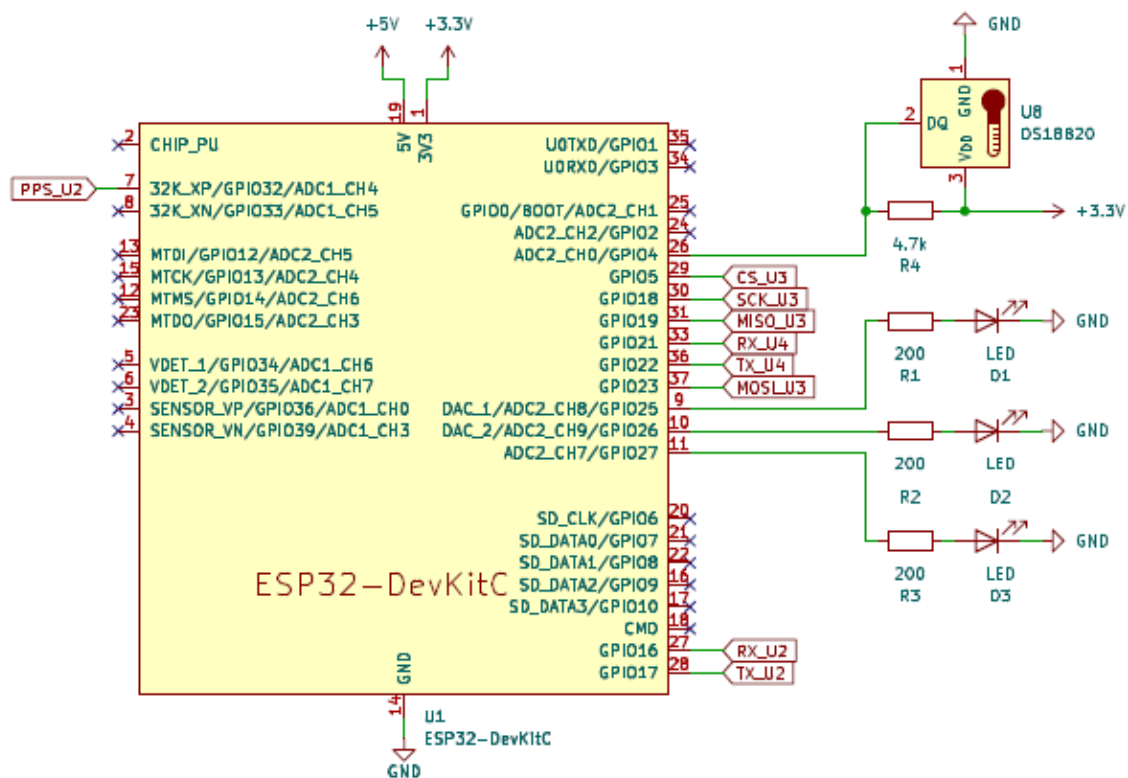
Kluczową funkcjonalnością wyróżniającą zaprojektowany system jest możliwość zdalnej rekonfiguracji parametrów pracy poprzez mechanizm atrybutów współdzielonych (Shared Attributes). W sekcji "Settings" (Rys. 7) przygotowano formularz, który pozwala użytkownikowi na modyfikację zmiennych systemowych.

Rys. 4.7. Panel zdalnej konfiguracji urządzenia. Zmiana wartości w polach formularza skutkuje natychmiastową aktualizacją atrybutów wspólnych, które trafiają do ESP32.

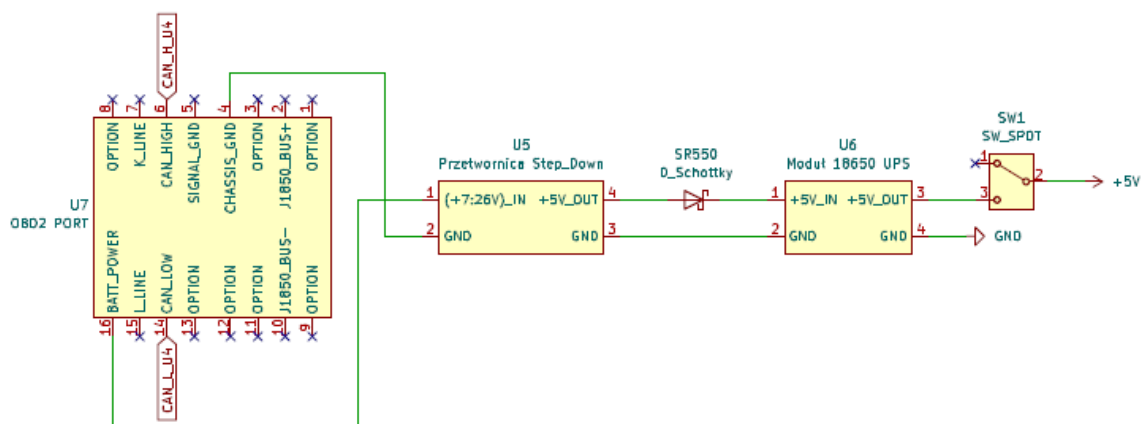
Zmiana wartości w tych polach powoduje wysłanie komunikatu aktualizacji danych do urządzenia. Oprogramowanie ESP32, dzięki zaimplementowanej funkcji `attributesCallback`, odbiera nowe nastawy i natychmiast dostosowuje cykl pracy koordynatora zadań, co pozwala na elastyczne zarządzanie zużyciem energii i częstotliwością próbkowania.

4.4 Wykonanie

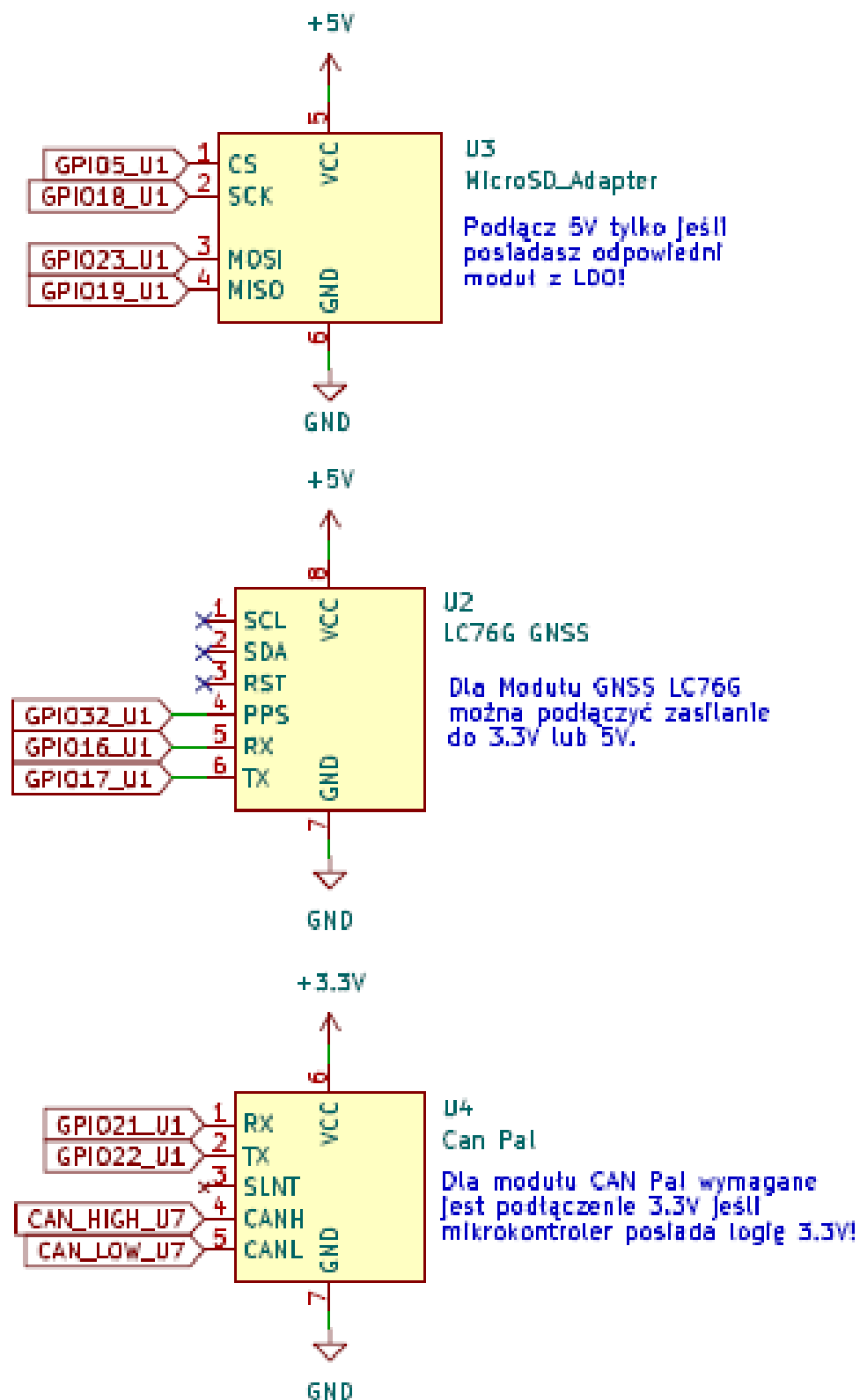
Proces realizacji urządzenia został podzielony na etap opracowania schematu połączeń elektrycznych (Rys 4.8., Rys.4.9., Rys.4.10.) oraz fizyczny montaż prototypu. Prototyp wykonano z wykorzystaniem płytki uniwersalnej, co pozwoliło na trwałe połączenie komponentów przy zachowaniu możliwości modyfikacji połączeń. Najpierw zamontowano główne moduły, a następnie całość uzupełniono o diody sygnalizacyjne LED (dla statusu Wi-Fi, GPS i zapisu SD), diodę Schottky'ego oraz przełącznik zasilania.



Rys. 4.8. Schemat połączeniowy ESP32

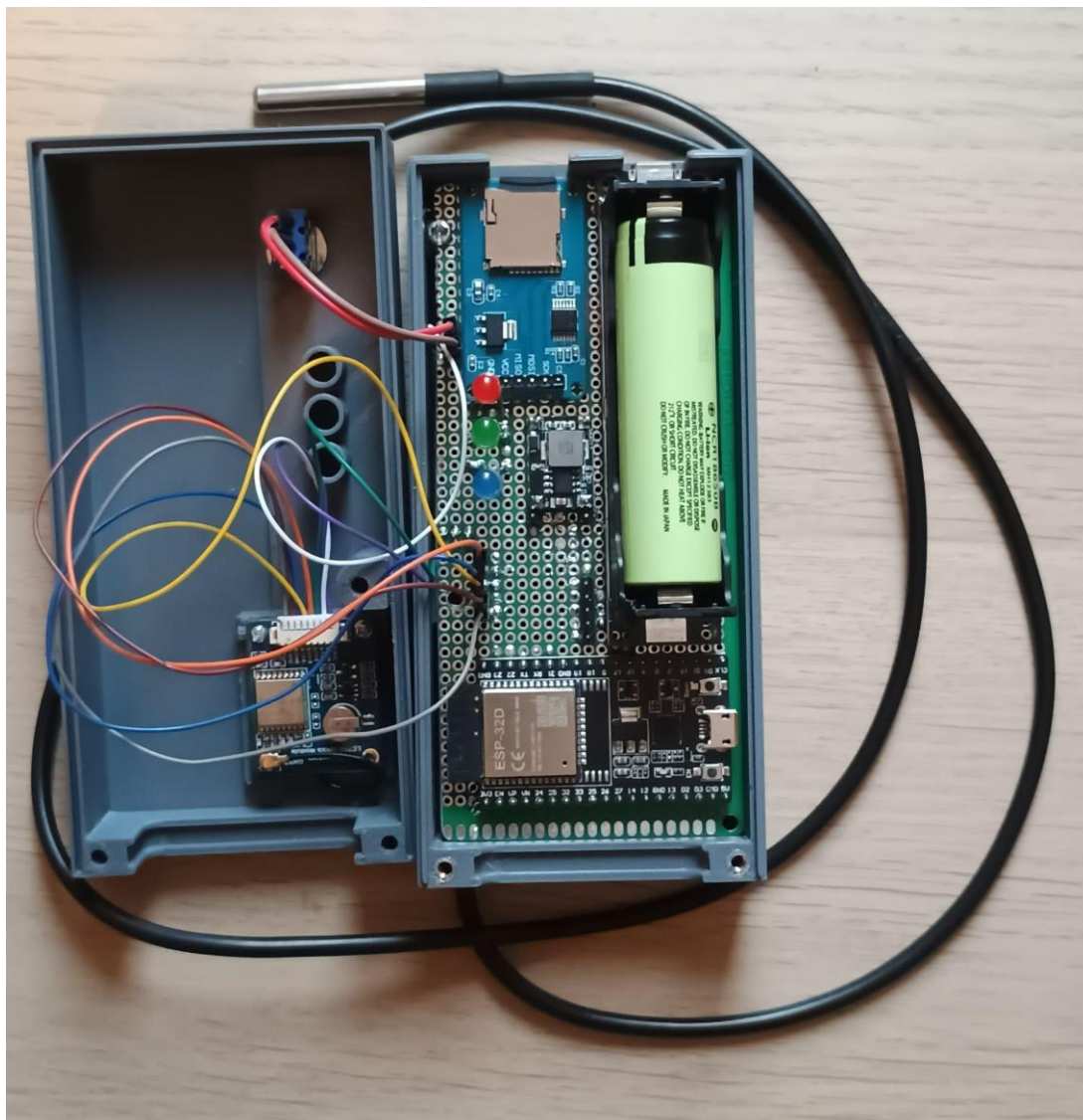


Rys. 4.9. Schemat połączeniowy zasilania



Rys. 4.10. Schemat połączeniowy peryferiów

Gotowy prototyp urządzenia przedstawiono na Rys.4.11. oraz Rys.4.12.



Rys. 4.11. Widok otwartego prototypu urządzenia. Widoczne osadzone ESP32 oraz moduły peryferyjne na płytce uniwersalnej.



Rys. 4.12. Urządzenie przygotowane do testów.

4.5 Testy

W celu potwierdzenia zgodności wykonanego urządzenia z założeniami projektowymi, przeprowadzono testy. Proces weryfikacji podzielono na testy laboratoryjne, obejmujące sprawdzenie podsystemów elektrycznych, oraz testy polowe, mające na celu ocenę pracy urządzenia w warunkach rzeczywistej eksploatacji w pojeździe.

4.5.1. Weryfikacja elektryczna i uruchomieniowa

Pierwszym etapem testów była kontrola poprawności montażu oraz stabilności zasilania. Przed instalacją mikrokontrolera w podstawce, zweryfikowano multimetrem napięcie wyjściowe fabrycznie skonfigurowanej przetwornicy Step-Down. Potwierdzono stabilną wartość 5 V, co jest napięciem bezpiecznym dla stabilizatora LDO znajdującego się na płycie rozwojowej ESP32.

Następnie zweryfikowano działanie układu podtrzymania zasilania (UPS). Symulowano nagłe odłączenie zasilania głównego (samochodowego), potwierdzając, że urządzenie płynnie przełącza się na zasilanie z ogniwa Li-Ion 18650, nie powodując przy tym resetu mikrokontrolera.

W kolejnym kroku przeprowadzono szczegółowe pomiary poboru prądu w dwóch charakterystycznych scenariuszach pracy, co pozwoliło oszacować czas autonomii urządzenia na zasilaniu awaryjnym:

1. **Scenariusz OFFLINE:** Symuluje pracę w pojeździe poza zasięgiem zaufanej sieci Wi-Fi. Urządzenie cyklicznie wybudza modem radiowy w celu skanowania sieci raz na 5 minut (pobór skacze do 160 mA), jednak większość czasu spędza na akwizycji danych i zapisie na kartę SD (50mA).
2. **Scenariusz ONLINE:** Symuluje pracę w zasięgu sieci. Urządzenie utrzymuje połączenie z brokerem MQTT, co generuje wyższy prąd bazowy (65 mA), a impulsy prądowe podczas wysyłki danych co 30 sekund (100 mA) są niższe niż podczas skanowania sieci w poprzednim przypadku. Jednak ich częstotliwość jest wyższa przez co średnia poboru prądu jest wyższa,

Szczegółowe wyniki pomiarów oraz szacowany czas pracy na akumulatorze przedstawiono w Tabeli 4.1.

Tabela 4.1. Bilans energetyczny urządzenia

Parametr	Scenariusz OFFLINE	Scenariusz ONLINE
Pobór Baza [mA]	50	65
Pobór Aktywny [mA]	80(Log) / 160 (WiFi Scan)	65(Log) / 100(Log/Send)
Średni prąd [mA]	~71.7	~89.5
Czas pracy (h)	~47.5 godziny	~38 godzin

4.5.2. Testy funkcjonalne oprogramowania

Testy funkcjonalne polegały na weryfikacji działania poszczególnych modułów programowych przy użyciu wbudowanych mechanizmów diagnostycznych, takich jak logi na porcie szeregowym oraz diody statusowe LED:

1. Sygnalizacja stanu: Zweryfikowano logikę sterowania diodami:
 - LED_WiFi (GPIO 27): Zapala się po poprawnym uzyskaniu adresu IP.
 - LED_GPS (GPIO 26): Sygnalizuje uzyskanie stabilnego "Fixa" (ważne dane lokalizacyjne).
 - LED_SD (GPIO 25): W stanie normalnym dioda ta świeci światłem ciągłym, sygnalizując poprawną inicjalizację karty i gotowość systemu plików. Zgaśnięcie diody informuje o błędzie krytycznym zapisu lub braku nośnika.
2. Symulacja awarii nośnika: Przeprowadzono test polegający na fizycznym wyjęciu karty microSD podczas pracy urządzenia. System prawidłowo wykrył brak nośnika (zgaszenie diody LED_SD i ustawienie flagi błędu w telemetry). Po ponownym włożeniu karty, mechanizm „SdModule::ensureReady”

automatycznie przywrócił sprawność systemu plików, wznowiając logowanie danych bez konieczności restartu zasilania.

3. Watchdog Timer: Sprawdzono odporność systemu na zawieszenie. Wymuszona pętla nieskończona w jednym z zadań (Tasks) spowodowała poprawną reakcję układu Watchdog (Task WDT) i restart systemu po upływie zdefiniowanego czasu (120 sekund).

4. Weryfikacja zarządzania zasobami (Heap & Stack Monitoring)

W celu potwierdzenia stabilności oprogramowania podczas pracy pod dużym obciążeniem, przeprowadzono monitorowanie zużycia pamięci operacyjnej RAM. Krytycznym scenariuszem testowym był moment synchronizacji danych z pliku buforowego (Pending), który wymaga jednoczesnej obsługi systemu plików (odczyt z karty SD) oraz transmisji sieciowej (obsługa stosu TCP/IP).

Poniższy log z portu szeregowego przedstawia zrzut diagnostyczny z portu szeregowego zarejestrowany podczas trwania procesu synchronizacji.

```
----- MEMORY STATUS -----  
[HEAP] Free: 109972 bytes, Min Free: 93592 bytes  
[TASK] Coordinator: 2336 bytes free stack  
[TASK] WiFi:      2184 bytes free stack  
[TASK] DataSync:  3624 bytes free stack  
[TASK] GPS:       2044 bytes free stack  
[TASK] Temp:      2084 bytes free stack  
[TASK] WebServer: 2572 bytes free stack  
[TASK] Sleep:     1524 bytes free stack  
-----
```

Analiza logów pozwala na sformułowanie następujących wniosków:

1. Brak wycieków pamięci: Wartość wolnej sterty ([HEAP] Free) utrzymuje się na poziomie ok. 109 kB, a najniższa zarejestrowana wartość (Min Free) wynosi ponad 93 kB, co stanowi bezpieczny margines dla układu ESP32.
2. Bezpieczeństwo wątków: Każde z zadań FreeRTOS posiada zapas wolnej pamięci stosu (free stack) wynoszący minimum 1500 bajtów. Oznacza to, że rozmiary stosów zadeklarowane w kodzie źródłowym zostały dobrane poprawnie

i nie występuje ryzyko przepełnienia stosu (Stack Overflow), co mogłoby prowadzić do niekontrolowanego restartu urządzenia.

5. Analiza zajętości pamięci masowej: Przeprowadzono analizę wielkości generowanych plików danych w formacie JSON Lines (.jsonl). Na podstawie pomiarów ustalono, że pojedynczy rekord telemetryczny (zawierający znacznik czasu, współrzędne, prędkość, temperaturę i statusy) zajmuje średnio 208 bajtów.

Przy założeniu wykorzystania karty pamięci o pojemności użytkowej 29,1 GB oraz ciągłej pracy urządzenia z domyślnym interwałem zapisu wynoszącym 15 sekund, przeprowadzono szacunkowe obliczenia czasu zapełnienia nośnika:

Pojemność karty: $29,1[GB] \approx 3,12 * 10^{10}[B]$

Rozmiar rekordu: $208[B]$

Maksymalna liczba rekordów: $\frac{3,12 * 10^{10}}{208} \approx 150\,000\,000$

Częstotliwość zapisu: 1 na 15[s]

Liczba rekordów na dobę $4 * 60 * 24 = 5760$

Szacowany czas pracy do zapełnienia karty:

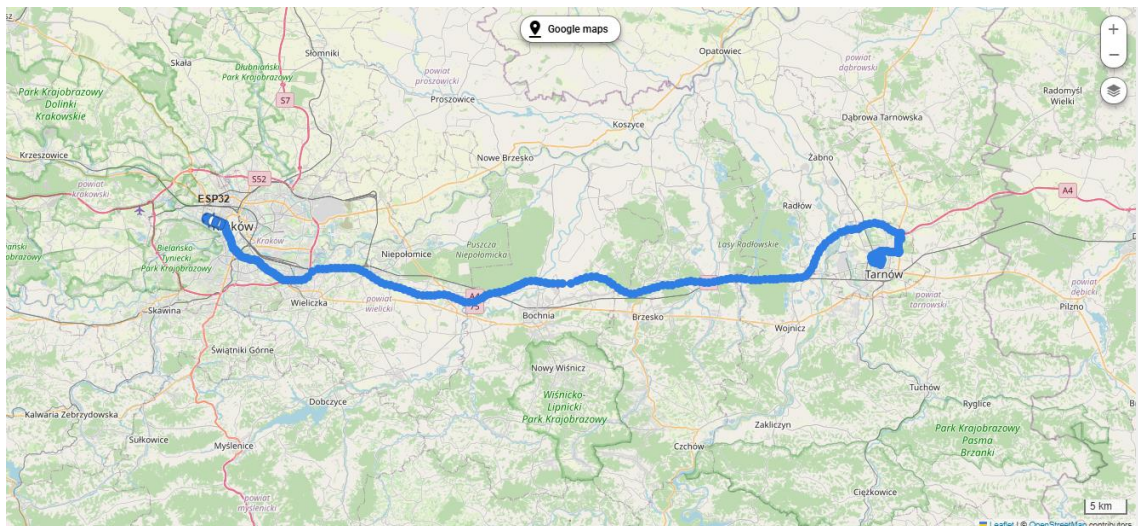
$$\frac{150\,000\,000}{5760} \approx 26\,041 \approx 71 \text{ lat}$$

Powyższe obliczenia wskazują, że przy przyjętym formacie danych i interwale próbkowania, pojemność karty SD jest w praktyce niewyczerpywalna w całym cyklu życia urządzenia, nawet przy uwzględnieniu narzutu systemu plików (FAT32).

4.5.3. Testy polowe i ciągłość danych (Scenariusz Offline/Online)

Kluczowym elementem weryfikacji był test drogowy mający na celu sprawdzenie mechanizmu buforowania danych. Przeprowadzono przejazd testowy na trasie o długości ok. 80 km (Rys. 4.13.). Procedura testowa:

1. Uruchomienie urządzenia w zasięgu sieci (Synchronizacja czasu, wysyłka próbna).
2. Celowe wyłączenie punktu dostępowego Wi-Fi na czas jazdy.
3. Weryfikacja zachowania systemu: Urządzenie, nie mogąc połączyć się z brokerem MQTT, zaczęło zapisywać dane do pliku buforowego pending.jsonl na karcie SD.
4. Ponowne włączenie sieci Wi-Fi.
5. Po odzyskaniu połączenia, zadanie TaskDataSync automatycznie wykryło dostępność sieci i przesłało zaległe rekordy z pliku pending do platformy ThingsBoard, zachowując chronologię zdarzeń.

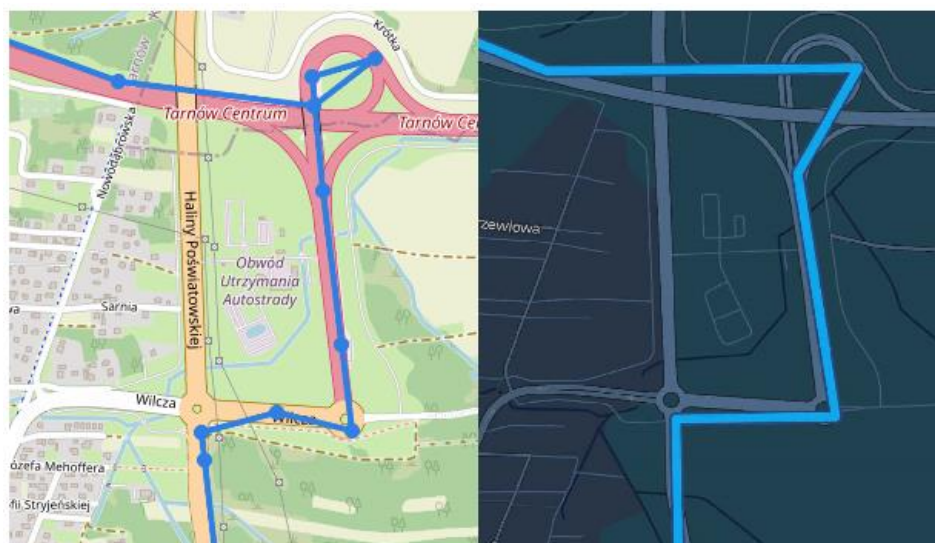


Rys. 4.13. Trasa zarejestrowana poprzez prototyp urządzenia i wyświetlona na platformie ThingsBoard

4.5.4. Ocena jakości danych GNSS

Dokładność modułu LC76G oceniono poprzez porównanie zarejestrowanej ścieżki (wygenerowanej na podstawie logów .jsonl) z trasą referencyjną wyznaczoną na mapie cyfrowej. Analiza wykazała, że w otwartym terenie błąd pozycjonowania w większości przypadków nie przekraczał 5 metrów. Natomiast zauważono również pojedyncze przypadki w których błąd pozycji wynosił nawet 30 metrów. Zauważono również poprawną pracę algorytmu "Cold Start" - czas uzyskania pierwszej pozycji (TTFF) po

wybudzeniu urządzenia wynosił średnio poniżej 30 sekund, co jest wartością zgodną z dokumentacją modułu.



Rys. 4.14. Porównanie wycinka trasy zarejestrowanego w ThingsBoard (po lewej) oraz lokalizację w Google Maps (po prawej)

4.5.5. Wizualizacja i telemetria

Ostatnim etapem była weryfikacja poprawności danych prezentowanych na platformie ThingsBoard oraz na lokalnym serwerze WWW urządzenia. Potwierdzono, że atrybuty takie jak prędkość (vel), wysokość (alt) oraz temperatura (temp) są poprawnie dekodowane z formatu JSON i aktualizowane w czasie rzeczywistym (przy dostępnym połączeniu sieciowym).

5. Podsumowanie i wnioski

W ramach niniejszej pracy dyplomowej zaprojektowano, wykonano i przetestowano prototyp systemu lokalizacji i telemetrii pojazdu, oparty na mikrokontrolerze ESP32 oraz systemie operacyjnym czasu rzeczywistego FreeRTOS. Głównym celem projektu było stworzenie rozwiązania modułowego, które łączyłoby funkcjonalność rejestratora parametrów jazdy z możliwością bezprzewodowej synchronizacji danych z chmurą IoT.

Zrealizowany prototyp spełnia wszystkie założenia funkcjonalne:

1. Akwizycja danych: System poprawnie interpretuje strumień NMEA z modułu GNSS oraz odczytuje ramki z magistrali CAN, integrując te dane w spójne rekordy telemetryczne.
2. Niezawodność zapisu: Zaimplementowany mechanizm wysyłki oraz zapisu skutecznie buforuje dane na karcie pamięci w przypadku braku łączności, a następnie automatycznie synchronizuje je z serwerem ThingsBoard po powrocie do bazy.
3. Architektura oprogramowania: Wykorzystanie RTOS pozwoliło na równoległą obsługę zadań krytycznych czasowo (CAN, GPS) oraz zadań o wysokim czasie wykonania (zapis na SD, obsługa stosu TCP/IP).

5.1. Napotkane problemy i ograniczenia realizacyjne

W toku prac projektowych i wdrożeniowych zidentyfikowano szereg wyzwań technicznych oraz ograniczeń, które wpłynęły na ostateczny kształt prototypu. Poniżej omówiono kluczowe trudności napotkane podczas realizacji pracy.

1. Ograniczenia platformy serwerowej (ThingsBoard Demo)

Istotnym wyzwaniem okazały się limity narzucone przez wersję demonstracyjną (Community Edition) platformy ThingsBoard. Zidentyfikowano ograniczenie przepustowości przyjmowania danych telemetrycznych, które uniemożliwiło przesyłanie dużych paczek danych w jednym komunikacie MQTT.

- Problem: Próba wysłania tablicy JSON zawierającej więcej niż 2 rekordy pomiarowe skutkowała odrzuceniem połączenia przez brokera lub utratą pakietów.
- Rozwiązanie: Wymusiło to programowe ograniczenie parametru BATCH_SIZE w oprogramowaniu układowym. System został skonfigurowany tak, aby dzielić dane historyczne (zbuforowane na karcie SD) na mniejsze fragmenty, co wydłużyło czas synchronizacji danych po powrocie urządzenia do trybu online, ale zapewniło stabilność transmisji.

2. Kompatybilność z nowoczesnymi systemami bezpieczeństwa pojazdów (Secure Gateway)

Podczas prób integracji z nowszymi modelami pojazdów (wyprodukowanymi po 2017 roku) napotkano barierę w postaci modułu SGW (Security Gateway).

- Problem: Nowoczesne architektury samochodowe blokują nieautoryzowany dostęp do magistrali CAN, uniemożliwiając odczyt parametrów oraz inżynierię wsteczną ramek bez posiadania certyfikatów producenta i dedykowanego sprzętu odblokowującego.
- Wniosek: Ostatecznie testy funkcjonalne ograniczono do pojazdów starszej generacji, co wskazuje na konieczność stosowania w przyszłości certyfikowanych bramek OBD2 typu passthru dla nowszych flot lub bezpośredniego podpięcia się do linii CAN przy np. silniku.

3. Wyzwania związane z miniaturyzacją (PCB)

Jednym z założeń projektowych było dążenie do miniaturyzacji urządzenia. Mimo opracowania schematu ideowego, w ramach niniejszej pracy nie udało się zrealizować dedykowanego obwodu drukowanego (PCB).

- Przyczyna: Ograniczenia czasowe i budżetowe wymusiły pozostanie przy konstrukcji prototypowej opartej na płycie uniwersalnej.
- Skutek: Urządzenie posiada większe gabaryty niż komercyjne odpowiedniki co stanowi główny obszar do usprawnienia w kolejnych iteracjach projektu.

5.2. Wnioski z realizacji

Analiza działania prototypu w warunkach laboratoryjnych i polowych pozwala na sformułowanie następujących wniosków:

- Wydajność platformy sprzętowej: Układ ESP32 dysponuje wystarczającą mocą obliczeniową (dwurdzeniowy procesor 240 MHz) do obsługi zaawansowanej logiki biznesowej, połączeń MQTT oraz obsługi systemu plików, pozostawiając margines zasobów na przyszłą rozbudowę.
- Model komunikacji Wi-Fi: Decyzja o wykorzystaniu Wi-Fi jako głównego medium transmisyjnego drastycznie obniżyła koszty eksploatacji (brak abonamentu GSM) i uprościła konstrukcję urządzenia. Ogranicza to jednak zastosowanie systemu do scenariuszy typu "czarna skrzynka" lub monitoringu flot powracających do bazy, wykluczając śledzenie antykradzieżowe w czasie rzeczywistym.
- Złożoność integracji CAN/OBD: Choć fizyczny odczyt ramek CAN działa poprawnie, pełna integracja ze standardem OBD2 (ISO 15765-4) wymaga skomplikowanej implementacji oraz dostosowania zapytań PID do specyfiki konkretnego producenta pojazdu.

5.3. Kierunki dalszego rozwoju

Otwarta architektura projektu oraz modułowa budowa kodu źródłowego stwarzają szerokie możliwości rozwoju. Poniżej przedstawiono kluczowe obszary, których implementacja pozwoliłaby na przekształcenie prototypu w pełni funkcjonalny produkt rynkowy.

Rozbudowa warstwy komunikacyjnej (IoT):

- Moduł LTE-M / NB-IoT: Zastąpienie lub uzupełnienie Wi-Fi modulem komórkowym (np. SIM7000 lub nRF9160) zapewniłoby ciągłą transmisję danych niezależnie od lokalizacji. Umożliwiłoby to realizację funkcji antykradzieżowych, takich jak powiadomienia "Push" o nieautoryzowanym ruchu pojazdu czy wyjeździe poza wyznaczoną geostrefę (Geofencing).

- Aktualizacje OTA (Over-The-Air): Implementacja mechanizmu zdalnej aktualizacji oprogramowania układowego przez Wi-Fi pozwoliłaby na naprawę błędów i dodawanie nowych funkcji bez konieczności fizycznego dostępu do urządzenia.
- Implementacja algorytmu cyklicznego uśpienia (Duty Cycling): Docelowe rozwiązanie powinno opierać się na automatycznej detekcji postoju. Proponuje się wdrożenie logiki, w której system po wykryciu braku przemieszczania się (na podstawie danych GNSS) przez okres 5 minut automatycznie przechodzi w stan głębokiego uśpienia (Deep Sleep). W celu zachowania łączności, układ będzie wybudzany cyklicznie co 10 minut dla wysłania pakietu statusowego, po czym ponownie przejdzie w stan uśpienia.
- Sprzętowy monitoring stanu baterii (ADC): W celu zwiększenia niezawodności pracy na zasilaniu akumulatorowym, układ należy rozbudować o dzielnik napięcia podłączony do przetwornika ADC mikrokontrolera ESP32. Umożliwi to ciągły pomiar napięcia ogniwa Li-Ion. Pozwoli to na bezpieczne zamknięcie systemu plików i zakończenie pracy urządzenia przed spadkiem napięcia poniżej poziomu krytycznego, co chroni akumulator przed głębokim rozładowaniem, a dane na karcie SD przed uszkodzeniem.
- Szyfrowanie pamięci masowej: W obecnej wersji pliki na karcie SD są zapisywane jawnym tekstem (JSON). Wdrożenie szyfrowania symetrycznego (np. AES-128) zabezpieczyłoby dane o trasach i parametrach pojazdu w przypadku kradzieży nośnika.
- Bezpieczna transmisja: Implementacja protokołu MQTTS (MQTT over SSL/TLS) z weryfikacją certyfikatów serwera zabezpieczyłaby transmisję przed atakami przy wysyłce.
- Pełny stos OBD2: Rozbudowa modułu CanModule o obsługę protokołu ISO-TP pozwoliłaby na odczyt kodów błędów (DTC - Diagnostic Trouble Codes) silnika oraz ich zdalne kasowanie.
- Analiza stylu jazdy: Wykorzystanie danych z akcelerometru oraz prędkości GPS do wykrywania gwałtownych hamowań, przyspieszeń i wchodzenia w zakręty, co jest kluczową funkcją w nowoczesnych systemach telematiki ubezpieczeniowej.

- Dedykowane PCB: Przeniesienie układu z płytki uniwersalnej na zaprojektowany obwód drukowany (PCB) z wykorzystaniem technologii montażu powierzchniowego (SMD) pozwoliłoby na znaczną miniaturyzację urządzenia i zwiększenie jego odporności na wibracje występujące w pojeździe.

6. Wykaz Publikacji

- [1] Adafruit Industries: *Adafruit CAN Pal - High-speed CAN transceiver. Dokumentacja techniczna modułu*. Dostępny: <https://learn.adafruit.com/adafruit-can-pal> (odwiedzona 03.12.2025).
- [2] Benoît B.: *ArduinoJson: Efficient JSON serialization for embedded C++*. Dostępny: <https://arduinojson.org/v7/> (odwiedzona 03.12.2025).
- [3] Burton M. et al.: *DallasTemperature Library. Biblioteka programistyczna*. Dostępny: <https://github.com/milesburton/Arduino-Temperature-Control-Library> (odwiedzona 03.12.2025).
- [4] Espressif Systems: *ESP32 Series Datasheet. Karta katalogowa układu, wersja 4.5. 2024.* Dostępny: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf (odwiedzona 03.12.2025).
- [5] Espressif Systems: *ESP32-DevKitC V4 User Guide. Dokumentacja płytki rozwojowej*. Dostępny: https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32/esp32-devkitc/user_guide.html (odwiedzona 03.12.2025).
- [6] Espressif Systems: *ESP-IDF Programming Guide: TWAI (Two-Wire Automotive Interface)*. Dostępny: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/twai.html> (odwiedzona 03.12.2025).
- [7] Hart M.: *TinyGPSPlus Library. Biblioteka programistyczna*. Dostępny: <https://github.com/mikalhart/TinyGPSPlus> (odwiedzona 03.12.2025).
- [8] O'Leary N.: *PubSubClient: A client library for MQTT messaging. Biblioteka programistyczna*. Dostępny: <https://github.com/knolleary/pubsubclient> (odwiedzona 03.12.2025).
- [9] Quectel: *LC76G GNSS Module Protocol Specification. Dokumentacja techniczna*. Dostępny: https://www.waveshare.com/wiki/LC76G_GNSS_Module (odwiedzona 03.12.2025).

[10] Stoffregen P. et al.: *OneWire Library. Biblioteka programistyczna*. Dostępny: <https://github.com/PaulStoffregen/OneWire> (odwiedzona 03.12.2025).

[11] ThingsBoard: *ThingsBoard Documentation & Guides. Dokumentacja systemu*. Dostępny: <https://thingsboard.io/docs/guides/> (odwiedzona 03.12.2025).

7. Załączniki

Załącznik 1. Repozytorium cyfrowe projektu Pełny kod źródłowy oprogramowania mikrokontrolera, schematy połączeń oraz pliki konfiguracyjne platformy ThingsBoard są dostępne w publicznym repozytorium w serwisie GitHub pod adresem: https://github.com/Mery-R/Engineering_Thesis

Instrukcja Użytkownika Systemu Lokalizacji Pojazdu

Niniejszy dokument opisuje sposób codziennego użytkowania urządzenia lokalizacyjnego, interpretację sygnalizacji świetlnej oraz obsługę panelu internetowego.

Spis treści

1. Uruchomienie i zasilanie
2. Sygnalizacja LED
3. Przełącznik zasilania
4. Panel internetowy
5. Tryb offline
6. Rozwiązywanie problemów

1. Uruchomienie i Zasilanie

Urządzenie jest zaprojektowane jako **bezobsługowe**.

1. **Montaż:** Umieść urządzenie w pojeździe w miejscu, które nie zasłania widoku nieba metalowymi elementami (np. na podszybiu lub desce rozdzielczej). Jest to kluczowe dla zasięgu GPS lub używasz CAN podłącz urządzenie do portu OBD2.
2. **Zasilanie:** Podłącz urządzenie do źródła prądu:
 - **W pojeździe:** Do portu OBD2.
 - **Stacjonarnie:** Do dowolnej ładowarki USB-C (np. od telefonu).
 - **Akumulatorowo:** Włóż akumulator do urządzenia.

Uwaga: Nie wolno podłączać jednocześnie USB-C oraz portu OBD2. Grozi to uszkodzeniem urządzenia.

3. **Start:** Urządzenie uruchomi się włączeniu przełącznika zasilania.

Zasilanie awaryjne (UPS)

Urządzenie posiada wbudowany akumulator. Po wyłączeniu zapłonu w samochodzie, lokalizator będzie kontynuował pracę przez pewien czas (zależny od poziomu naładowania).

2. Sygnalizacja LED (Co oznaczają diody?)

Na obudowie znajdują się trzy diody informujące o aktualnym stanie urządzenia.

Dioda (Kolor)	Stan	Oznaczenie	Co robić?
 WiFi (Niebieska)	Świeci	Połączono (Online)	System działa poprawnie, dane są wysyłane na żywo.
	Zgaszona	Brak sieci (Offline)	To normalne w trasie. Dane zapisują się na karcie SD.
 GPS (Zielona)	Świeci	Pozycja ustalona	Lokalizacja jest precyzyjna.
	Zgaszona	Szukanie satelitów	Poczekaj chwilę. W garażach podziemnych /tunelach brak sygnału jest normalny.
 SD (Czerwona)	Świeci	Karta OK	Wszystko w porządku, system plików działa.
	Zgaszona	Błąd Karty!	Wyjmij i włóż kartę SD ponownie. Jeśli nie pomaga, wymień kartę.

3. Przełącznik zasilania

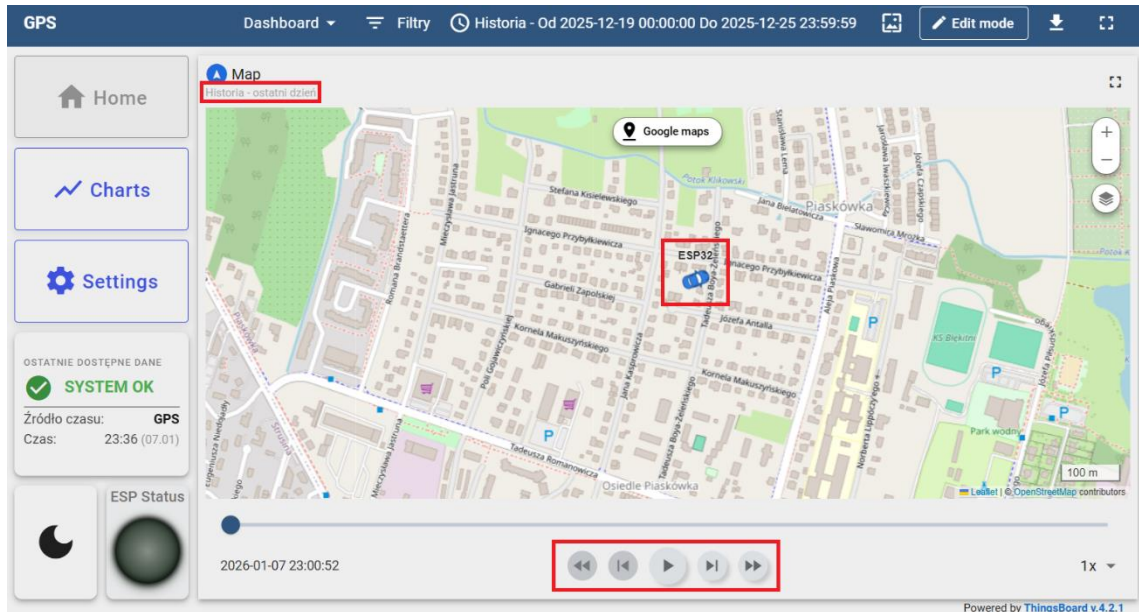
Urządzenie wyposażone jest w jeden przełącznik zasilania, który wyłącza urządzenie.

4. Obsługa Panelu Online (ThingsBoard)

Dostęp do danych lokalizacyjnych możliwy jest przez przeglądarkę internetową (na komputerze lub telefonie).

Widok Mapy (Home / Map)

Główny ekran systemu.

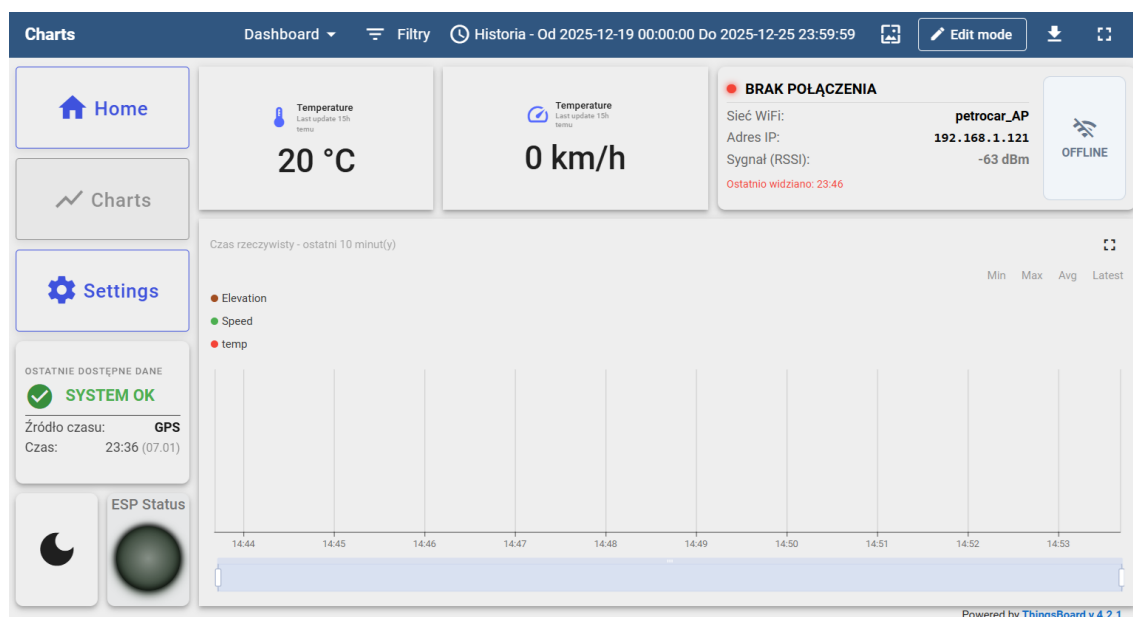


- **Znacznik:** Pokazuje ostatnią znaną pozycję pojazdu.
- **Linia:** Rysuje przebytą trasę.
- **Historia:** Użyj suwaka czasu na dole ekranu lub kalendarza w prawym górnym rogu, aby zobaczyć, gdzie pojazd był wczoraj lub tydzień temu.

Wykresy i Parametry (Charts)

Tutaj znajdziesz szczegółowe dane telemetryczne:

- **Prędkość:** Aktualna prędkość pojazdu (km/h).
- **Temperatura:** Odczyt z czujnika temperatury.
- **Sila Sygnału (RSSI):** Informuje o jakości połączenia z siecią WiFi.



Ustawienia (Settings)

W tej zakładce możesz zdalnie zmieniać parametry pracy urządzenia bez konieczności podłączania go do komputera.

- **Interwał zapisu (Delay):** Określa, co ile sekund urządzenie pobiera pozycję.
 - *Mniejsza wartość (np. 5s)* = Bardziej dokładna trasa, ale szybsze zużycie baterii/danych.
 - *Większa wartość (np. 60s)* = Oszczędność energii.
- **Rozmiar paczki (Batch Size):** Ile pomiarów jest wysyłanych naraz.

Uwaga: Zmiana ustawień zostanie wprowadzona w urządzeniu przy następnym połączeniu z serwerem.

The screenshot shows the 'Settings' page in the ThingsBoard interface. The top navigation bar is the same as the dashboard. The left sidebar has buttons for 'Home', 'Charts', and 'Settings'. The main area contains a form for configuring the ESP32 device. The form has four input fields: 'Number of records sent in one package*' with value 2, 'Minimal Batch size to trigger save*' with value 2, 'Delay between data acquisition (s)*' with value 15, and 'Delay between WiFi reconnections (s)*' with value 60. There is a checkbox labeled 'Require synchronized time' which is checked. At the bottom right, there are 'Reset' and 'Save' buttons. The footer says 'Powered by ThingsBoard v.4.2.1'.

5. Praca w trybie Offline (Brak zasięgu)

System jest odporny na zaniki zasięgu WiFi.

1. Gdy dioda **WiFi (Niebieska)** zgaśnie, urządzenie przechodzi w tryb rejestratora.
2. Wszystkie dane (trasa, prędkość) są zapisywane na karcie pamięci (bufor).
3. Po powrocie w zasięg znanej sieci WiFi, urządzenie automatycznie “dosyła” zaległe dane.
4. Historia trasy na mapie uzupełni się automatycznie.

6. Rozwiązywanie problemów

Problem: Urządzenie nie wysyła danych, świeci tylko czerwona i zielona dioda.

- **Rozwiązanie:** Urządzenie jest poza zasięgiem skonfigurowanej sieci WiFi. To normalne zachowanie. Dane zostaną wysłane po powrocie do bazy/domu.

Problem: Dioda GPS (Zielona) nie chce się zaświecić.

- **Rozwiązanie:** Upewnij się, że urządzenie “widzi” niebo. Sygnał GPS nie przenika przez betonowe stropy garaży czy gęste zadaszenia. Pierwsze ustalenie pozycji po długim wyłączeniu (Cold Start) może potrwać do 1 minut.

Problem: Dioda SD (Czerwona) zgasła.

- **Rozwiązanie:** Awaria zapisu. Sprawdź, czy karta microSD jest włożona poprawnie. Jeśli tak, sformatuj ją w komputerze (system plików FAT32) i włóż ponownie.